

PROMPT ENGINEERING FOR DEVELOPERS

SUCCINCTLY[®]

BY ED FREITAS

Prompt Engineering for Developers Succinctly

Ed Freitas

Foreword by Daniel Jebaraj



Copyright © 2026 by Syncfusion, Inc.

2501 Aerial Center Parkway

Suite 111

Morrisville, NC 27560

USA

All rights reserved.

ISBN: 978-1-64200-254-6

Important licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal or educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

SYNCFUSION, SUCCINCTLY, ESSENTIAL, ESSENTIAL STUDIO, BOLDDesk, BOLDSIGN, BOLD BI, and BOLD REPORTS are the registered trademarks of Syncfusion, Inc.

Technical Reviewer: James McCaffrey

Copy Editor: Jacqueline Bieringer, content producer, Syncfusion, Inc.

Acquisitions Coordinator: Graham High, content team lead, Syncfusion, Inc.

Proofreader: Naomi Spicer, content producer, Syncfusion, Inc.

Table of Contents

About the Author	8
Acknowledgments	9
Chapter 1 Introduction and Setup.....	10
Overview	10
What is prompt engineering?	10
Why do developers need prompt engineering?.....	10
The AI assistant as a pair programmer	11
Expectations: AI's role in development	11
The importance of the environment	12
Choosing your AI assistant	12
Understanding AI capabilities and limitations.....	13
Setup.....	13
Chapter 2 Prompting Essentials	20
Overview	20
Clarity, specificity, and context	20
Clarity	20
Specificity	21
Context.....	21
Role-playing and persona prompts	22
Iterative prompt refinement.....	23
Iteration 1: Initial prompt	23
Iteration 2: Refined prompt	25
Iteration 3: Further refined prompt	27
Output format specification	29

Chapter 3 Advanced Prompting	32
Overview	32
Few-shot prompting	32
How few-shot prompting works	32
Chain-of-thought prompting	34
Why CoT prompting is effective	34
Self-correction and debugging with AI	37
Integrating AI into your workflow	41
Practical integration strategies	41
Best practices for workflow integration	41
Chapter 4 Kick-Off: Database Design	42
Overview	42
Understanding the application requirements	42
Database design principles	42
Database design principles	43
Creating the database and tables	44
Initial data population prompts	48
Chapter 5 Core App: CRUD Operations	52
Overview	52
Understanding CRUD operations	52
Prompting for PHP database connection	52
Generating code for listing documents (read)	54
Retrieving all documents	54
Retrieving a single document by ID	61
Generating code for adding new documents (create)	65
Generating code for updating and deleting	70

Finished back-end code	76
Chapter 6 App Front End.....	83
Overview	83
Where we left off	83
Closing Thoughts	105

The *Succinctly* Series of Books

Daniel Jebaraj
CEO of Syncfusion, Inc.

When we published our first *Succinctly* series book in 2012, *jQuery Succinctly*, our goal was to produce a series of concise technical books targeted at software developers working primarily on the Microsoft platform. We firmly believed then, as we do now, that most topics of interest can be translated into books that are about 100 pages in length.

We have since published over 200 books that have been downloaded millions of times. Reaching more than 2.7 million readers around the world, we have more than 70 authors who now cover a wider range of topics, such as Blazor, machine learning, and big data.

Each author is carefully chosen from a pool of talented experts who share our vision. The book before you and the others in this series are the result of our authors' tireless work. Within these pages, you will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

We are absolutely thrilled with the enthusiastic reception of our books. We believe the *Succinctly* series is the largest library of free technical books being actively published today. Truly exciting!

Our goal is to keep the information free and easily available so that anyone with a computing device and internet access can obtain concise information and benefit from it. The books will always be free. Any updates we publish will also be free.

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctlyseries@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Please follow us on social media and help us spread the word about the *Succinctly* series!



About the Author

Ed Freitas is a business process automation consultant, technologist, and solutions architect with a focus on customer success. Since 2024, Ed has run Copiloted, an AI automation agency that helps organizations of all sizes use AI effectively and automate critical business processes, increasing their ROI and reducing costs: <https://getcopiloted.com>.

He enjoys technology, learning, playing soccer, running, traveling, and spending time with his family.

Ed can also be found at <https://edfreitas.me>.

Acknowledgments

Thank you to the fantastic [Syncfusion](#) team that helped this book become a reality—especially Jacqueline Bieringer, Tres Watkins, and Graham High.

The manuscript manager and technical editor thoroughly reviewed the book’s organization, code quality, and overall accuracy—Graham High from Syncfusion and [James McCaffrey](#). Thank you both.

I dedicate this book to my children—may your journeys be blessed.

Chapter 1 Introduction and Setup

Overview

Let's begin by identifying what this book covers and what it doesn't. This book is not a [vibe coding](#) tutorial, nor is it specific to any tool, platform, or [large language model](#).

The goal of this book is to showcase how to write prompts effectively to assist with coding, independent of the platform, tool, or model used. It's more about the process of learning to prompt effectively, rather than the final result of creating a full-blown application.

We'll be using the free version of [ChatGPT](#), but you can use any other tool, platform, or model. Just be aware that the results you get might differ significantly from mine if you use another tool. I'll use the terms *AI assistant* and *ChatGPT* interchangeably.

The complete source code used in this book is available in this GitHub [repository](#).

What is prompt engineering?

[Prompt engineering](#) is an emerging discipline focused on designing, refining, and optimizing inputs (prompts) for [large language models](#) (LLMs) to achieve desired outputs. It's the art and science of communicating effectively with AI, transforming abstract goals into concrete instructions that an AI can execute.

As AI models become more sophisticated, the ability to craft effective prompts becomes a critical skill for developers, enabling them to harness the full potential of these powerful tools.

At its core, prompt engineering involves understanding how LLMs process information and respond to various types of input. It's not just about asking a question; it's about structuring that question, providing context, specifying desired formats, and guiding the AI's thought process to produce accurate, relevant, and high-quality results.

Prompt engineering is dynamic, constantly evolving with new models and techniques, making continuous learning essential for all practitioners, from citizen developers to experienced practitioners.

Why do developers need prompt engineering?

For developers, prompt engineering is rapidly becoming an indispensable skill. Modern software development often involves more than just writing code; it includes design, testing, documentation, and problem-solving. When prompted effectively, AI assistants can augment every stage of this process, acting as intelligent copilots. Developers can leverage prompt engineering to:

- **Accelerate code generation:** Generate boilerplate code, functions, or even entire scripts in various programming languages, significantly reducing development time.
- **Improve code quality:** Obtain suggestions for refactoring, optimization, and adherence to best practices.
- **Automate testing:** Generate unit tests, integration tests, and test data based on code descriptions or requirements.
- **Streamline documentation:** Create API documentation, user manuals, and inline comments directly from code or specifications.
- **Debug and troubleshoot:** Get assistance in identifying errors, understanding complex error messages, and finding potential fixes.
- **Learn new technologies:** Rapidly grasp new frameworks, languages, or concepts by asking the AI for explanations, examples, and comparisons.
- **Design and architect:** Brainstorm architectural patterns, database schemas, and UI or UX flows.

By mastering prompt engineering, developers can transform their workflow, moving from merely coding to orchestrating complete systems, thereby enhancing productivity and innovation.

The AI assistant as a pair programmer

Think of your AI assistant not as a replacement for a developer, but as an incredibly knowledgeable and tireless pair programmer. This digital partner can handle repetitive tasks, offer alternative solutions, and provide instant access to a vast repository of programming knowledge.

The key to a successful partnership lies in effective communication, which is precisely where prompt engineering comes in. In a traditional pair programming setup, two developers collaborate at a single workstation, discussing approaches, reviewing code, and catching errors in real time. An AI assistant can replicate many of these benefits:

- **Real-time feedback:** Get immediate suggestions for code improvements or alternative implementations.
- **Knowledge augmentation:** Access information on obscure APIs, complex algorithms, or best practices without leaving your IDE.
- **Task offloading:** Delegate routine coding tasks, so you can focus on higher-level design and problem-solving.
- **Error detection:** Proactively identify potential bugs, security vulnerabilities, or performance bottlenecks.

This beneficial relationship allows developers to be more efficient, creative, and less prone to burnout, ultimately leading to better business solutions.

Expectations: AI's role in development

While AI assistant capabilities are impressive, it's crucial to set realistic expectations. AI is a tool, albeit a powerful one, and not a sentient entity. It excels at pattern recognition, code generation, and information synthesis, but it lacks accurate understanding, creativity, and critical judgment in the human sense.

Key expectations to maintain:

- **AI is a copilot, not an autonomous driver:** Always review and verify AI-generated code and suggestions. The AI can make mistakes, introduce subtle bugs, or provide suboptimal solutions.
- **Garbage in, garbage out:** The quality of the AI's output is directly proportional to the quality of the prompt. Poorly formulated prompts will yield poor results.
- **Context is king:** AI models operate based on the context they are given. Without sufficient context, their responses may be generic or irrelevant.
- **Security and ethics:** Be mindful of sensitive information and intellectual property when using AI. Always consider the ethical implications of AI-generated content, especially in production systems.
- **Continuous learning:** The field of AI and prompt engineering is evolving rapidly. Staying up-to-date with new techniques and model capabilities is vital to maximizing its utility.

The importance of the environment

To practice prompt engineering for development effectively, you need a well-configured environment that includes both your chosen AI assistant and your local development setup. Although there is no defined rule on what the ideal environment is, it must work well with the programming language you will be using for your project.

Choosing your AI assistant

The landscape of AI assistants is rapidly evolving, with several powerful models available, each with its own strengths and nuances. For this book, we will focus on general principles applicable across different LLMs, but it's beneficial to understand the characteristics of some popular choices:

- **Gemini (Google):** Multimodal capabilities, strong reasoning, code generation, integration with Google services. It is best suited for complex problem solving, creative code generation, and diverse output formats.
- **ChatGPT (OpenAI):** Highly conversational, excellent for brainstorming, code snippets, and explanations. It is best suited for general coding tasks, quick answers, and learning new concepts.
- **Claude (Anthropic):** Focus on safety and helpfulness, long context windows, strong ethical guidelines. It is best suited for sensitive applications, detailed documentation, and long-form code generation.
- **GitHub Copilot (OpenAI/GitHub):** IDE-integrated, real-time code suggestions, autocompletion. Rapid code writing, boilerplate generation, and learning syntax.

When choosing an AI assistant, consider factors such as its code-generation capabilities, context window size (how much information it can retain from previous uses), ease of integration into your workflow, and cost. For hands-on learning, starting with a free or easily accessible model, such as the public versions of Gemini or ChatGPT, is recommended. Another option to consider is [Grok](#).

Understanding AI capabilities and limitations

While AI assistants are incredibly powerful, they are not infallible. Understanding their capabilities and, more importantly, their limitations is crucial for practical prompt engineering and avoiding frustration. Let's look at these capabilities and limitations.

Capabilities:

- **Code generation:** Can produce code snippets, functions, classes, and even entire application structures in various languages.
- **Debugging assistance:** Can help identify errors, explain error messages, and suggest fixes.
- **Refactoring and optimization:** Can propose ways to improve code readability, performance, and maintainability.
- **Documentation:** Can generate comments, docstrings, and API documentation.
- **Learning and explanation:** Can explain complex concepts, algorithms, and design patterns.
- **Translation:** Can translate code between programming languages or natural languages.

Limitations:

- **Lack of true understanding:** AI models do not truly understand the code they generate. They are pattern-matching machines, which can lead to subtle bugs or nonsensical code.
- **Hallucinations:** AI models can sometimes generate plausible-sounding but incorrect information, including nonexistent functions or APIs.
- **Security vulnerabilities:** AI-generated code may contain security flaws. Always review it carefully, especially for applications handling sensitive data.
- **Outdated knowledge:** The training data for AI models has (sometimes) a cutoff date, so they may not be aware of the latest libraries, frameworks, or security patches.
- **Context dependency:** The quality of the output depends heavily on the prompt and the provided context.

By keeping these capabilities and limitations in mind, you can use AI as a powerful assistant while maintaining your role as the ultimate arbiter of code quality and correctness.

Setup

Throughout this book, we'll use the free version of ChatGPT to demonstrate how to write a [PHP](#) script that uses a small SQLite database. The reason I've chosen PHP rather than a trendier framework like [Next.js](#) is that PHP is available on any web host and powers most of the internet. Even though the examples will revolve around PHP, the point is not to focus on PHP, but instead on the prompts and process for interacting with the LLM.

To get started, download the [XAMPP](#) PHP development environment. Choose the version that corresponds to your operating system. In my case, I'll be using Windows 11.



Figure 1-a: XAMPP Website

Once you have downloaded the XAMPP installer, run it as an administrator. In Windows, right-click the installation file.

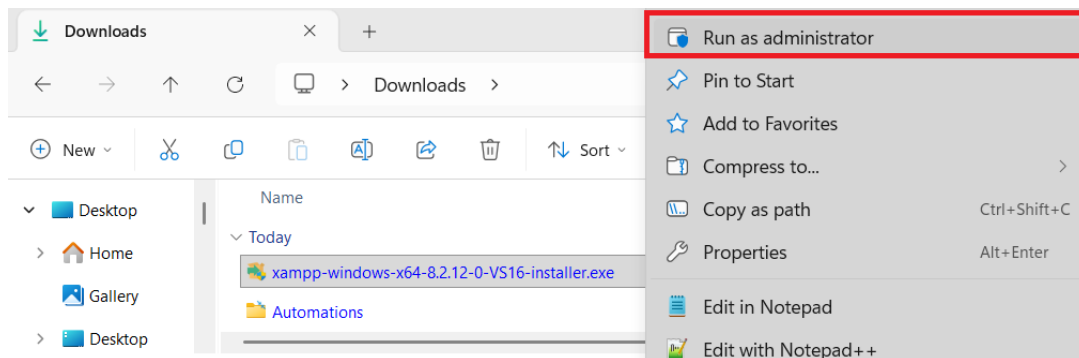


Figure 1-b: XAMPP Installer File (Run as administrator)

After executing the installer, you might see this warning pop up. For this book, it's not relevant, so click **OK**.

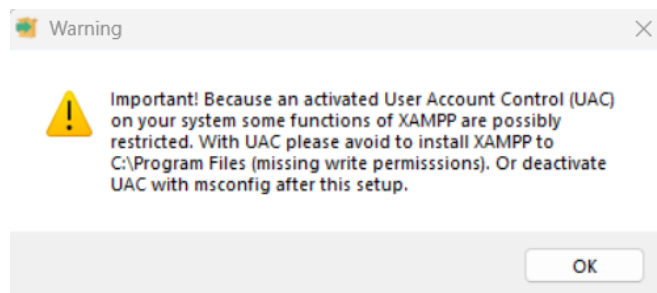


Figure 1-c: Warning: User Account Control (UAC)

Then, you'll see the following screen. Click **Next** to continue.

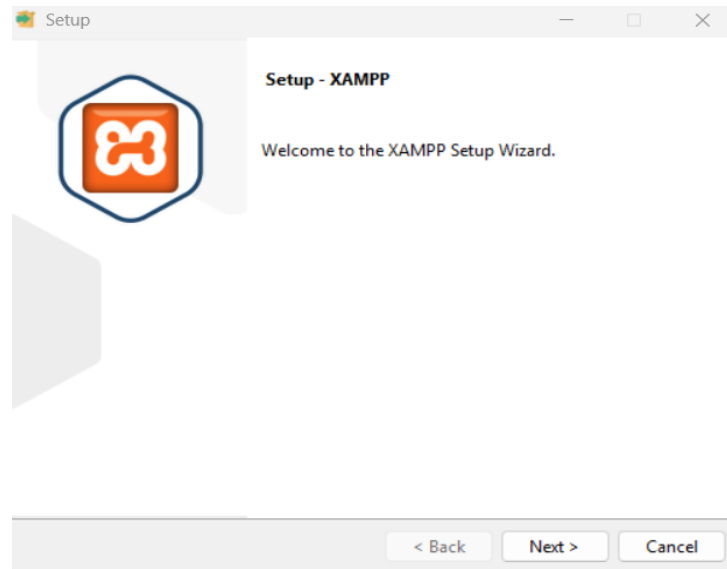


Figure 1-d: XAMPP Installation (Setup—XAMPP)

You'll see all the components that can be installed during the XAMPP installation. By default, all components are selected, so let's unselect all options except MySQL and phpMyAdmin. Once done, click **Next**.

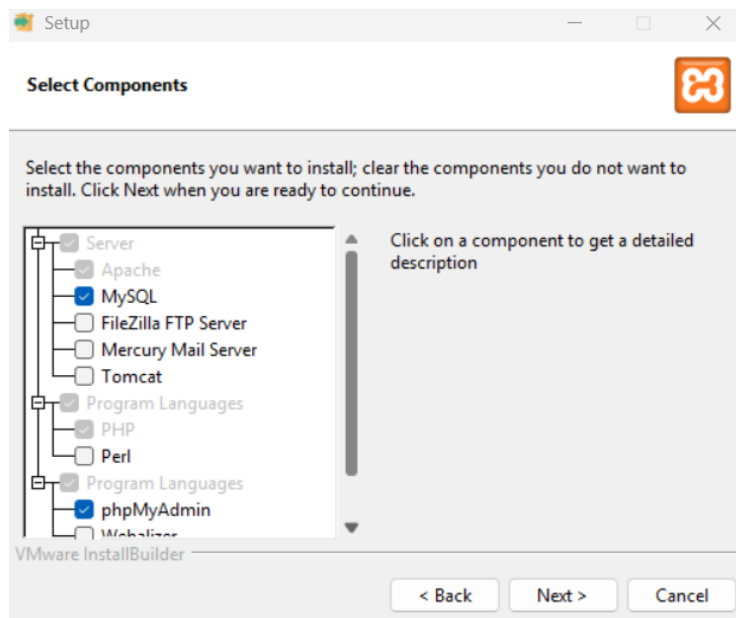


Figure 1-e: XAMPP Installation (Select Components)

Now, we can select the installation folder. By default, the installer uses C:\xampp as the installation folder name on Windows. I suggest you leave it as is. Click **Next** to continue.

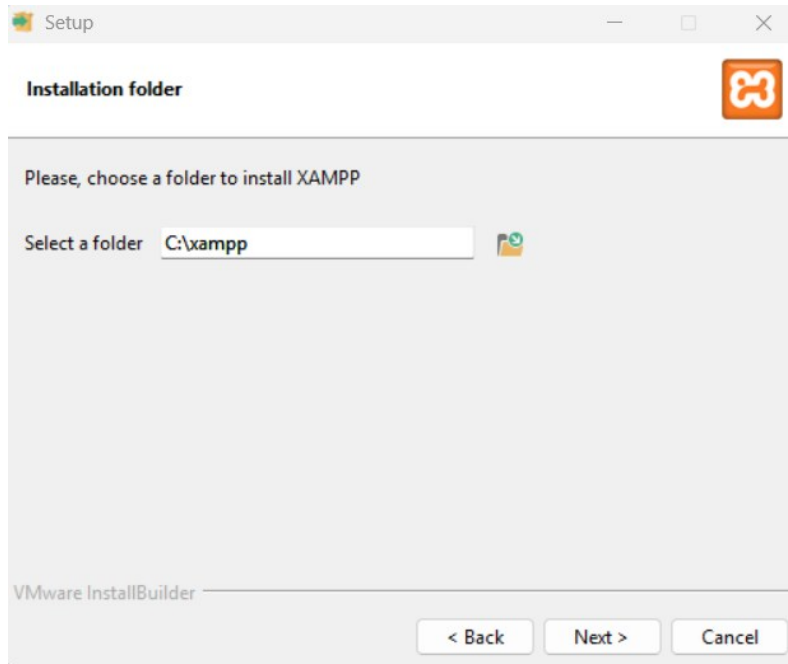


Figure 1-f: XAMPP Installation (Installation folder)

After that, we can see the option to select the installation language, which is set to English by default. The installation is also available in German (Deutsch). Click **Next** to continue.

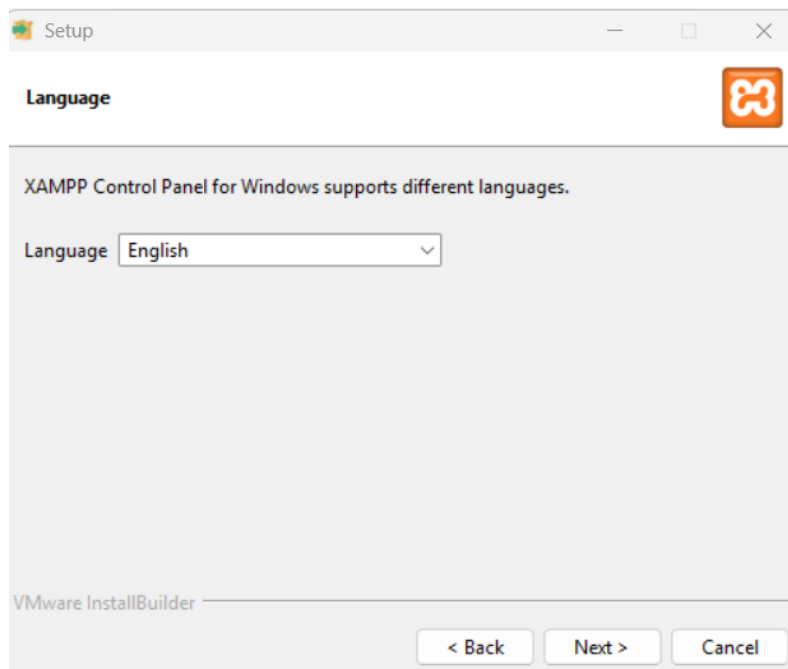


Figure 1-g: XAMPP Installation (Language)

At this stage, we are almost ready to start the installation. So, click **Next** to continue.

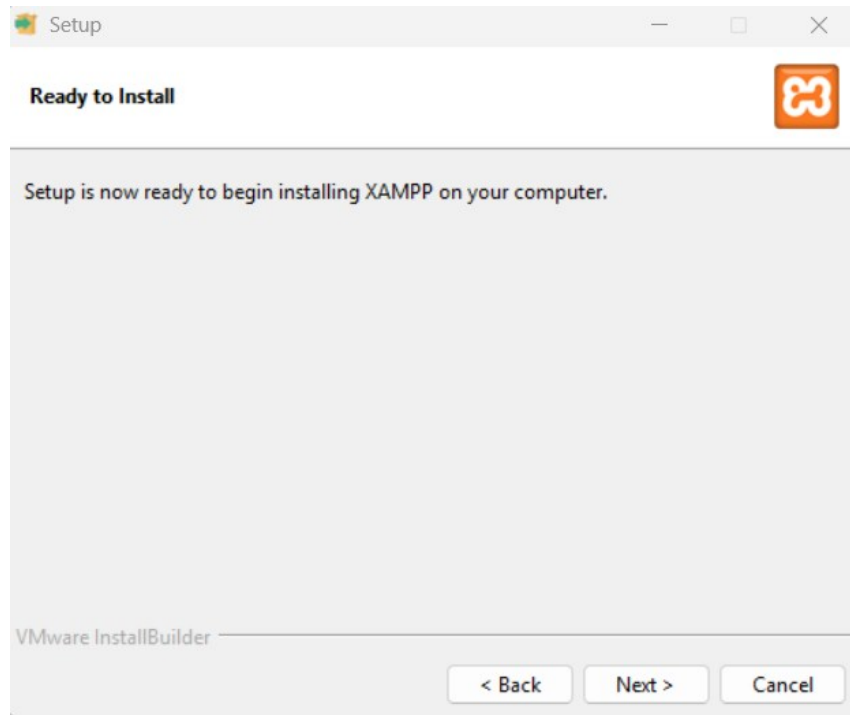


Figure 1-h: XAMPP Installation (Ready to Install)

Alright—so, one final **Next** button to click to install XAMPP.

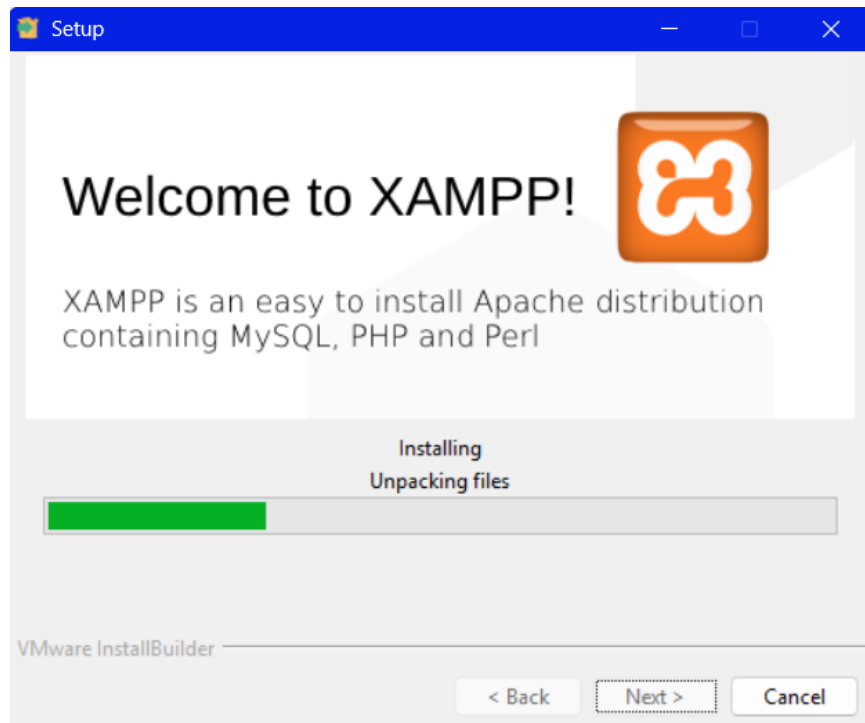


Figure 1-i: XAMPP Installation (Installing)

Once the installation has finished, you can open the XAMPP control panel directly.

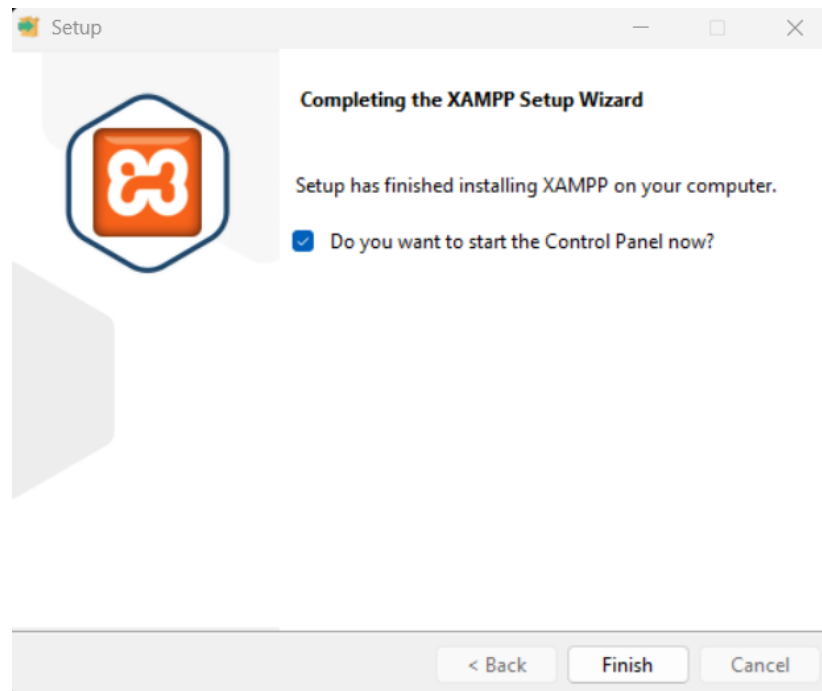


Figure 1-j: XAMPP Installation (Completed)

Once we've reached the final screen, click **Finish**.

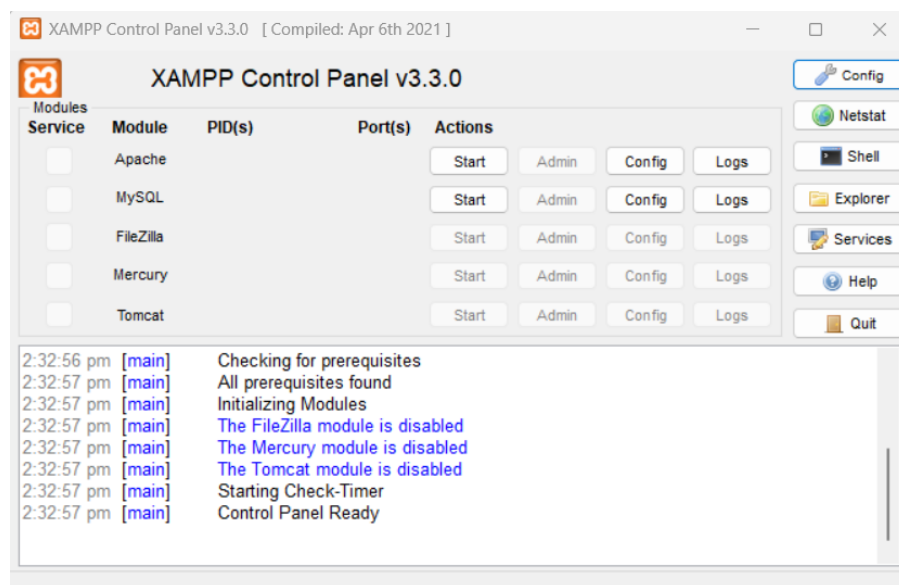


Figure 1-k: XAMPP Control Panel

Click the **Start** buttons next to the Apache and MySQL labels. This will start the Apache web server and the MySQL database locally. You'll see the following screen.

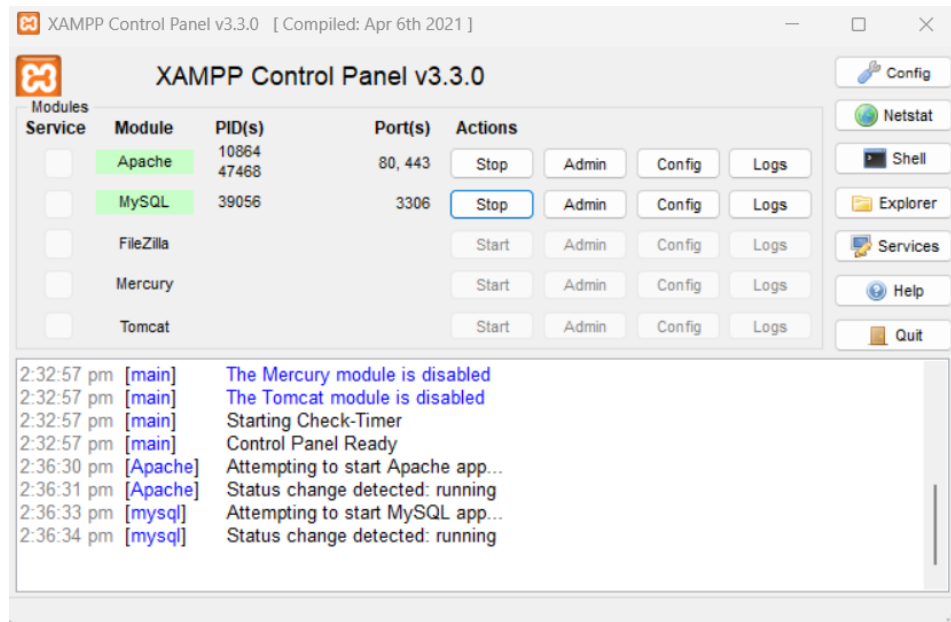


Figure 1-l: XAMPP Control Panel (Apache and MySQL Started)

When working locally with XAMPP, you'll need to place your PHP source files in the root of the local Apache web server folder. You can do this by clicking the **Explorer** button and placing the files within the **htdocs** folder.

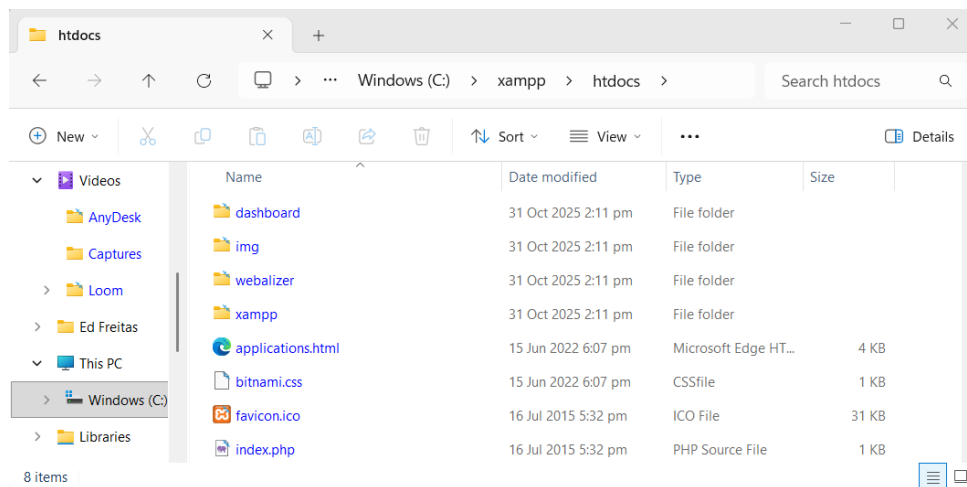


Figure 1-m: The Local Apache Web Server Root Folder (htdocs)

As you can see, this folder already contains some demo files. Excellent. Now that we have XAMPP up and running, let's start learning how to do effective prompting.

Let's first cover some prompting techniques that will come in handy later.

Chapter 2 Prompting Essentials

Overview

Having set up your development environment and understood the capabilities and limitations of AI, the next crucial step is to master the basics of crafting effective prompts.

This chapter delves into the foundational principles that transform vague requests into precise instructions, enabling AI to deliver high-quality, relevant outputs for quick development tasks.

Clarity, specificity, and context

The cornerstones of practical prompt engineering are clarity, specificity, and context. These three elements work in concert to ensure the AI understands your intent without ambiguity, leading to more accurate and helpful responses.

Clarity

Clarity means using straightforward language that the AI can easily interpret. Avoid jargon where simpler terms suffice and structure your sentences to be unambiguous. If a human could misinterpret your request, an AI likely will, too.

Let's begin with a poor and unclear prompt.

Code Listing 2-a: Poor (Unclear) Prompt Example

```
Make a database for my app.
```

This prompt is unclear. What kind of app? What database technology? What tables are needed? Here, the AI has too many unknowns to provide a helpful response.

Now, let's look at a clearer prompt.

Code Listing 2-b: Clearer Prompt Example

```
I need a database schema for a document-expiration-tracking application.  
  
The application will store information about documents like driver's  
licenses and passports, including their expiration dates.
```

This prompt is clearer because it states the application's purpose, giving the AI a better understanding of the domain.

Specificity

Specificity, on the other hand, involves providing precise details about what you want the AI to do, how it should do it, and what the expected output should look like. The more specific you are, the less the AI has to guess, reducing the chances of irrelevant or incorrect outputs.

Let's look at a nonspecific prompt.

Code Listing 2-c: Another Poor Prompt Example (Not Specific Enough)

```
Write some PHP code for a form.
```

This prompt is too generic. What kind of form? What fields? What should the form do?

Now, let's look at a more specific prompt example.

Code Listing 2-d: Specific Prompt Example

```
Write PHP code for an HTML form that allows users to add a new document.  
The form should have input fields for:
```

```
'Document Name' (text),  
'Document Type' (dropdown with options: 'Driver's License', 'Passport',  
'ID Card'),  
'Expiration Date' (date input),  
and 'Notes' (textarea).
```

```
Include basic HTML structure and a submit button.
```

Here, we've specified the form's purpose, the exact fields and their types, and even some fundamental UI elements. This level of detail guides the AI to generate highly relevant code.

Context

Context provides the background information the AI needs to understand the broader picture of your request. This can include previous conversations, project requirements, existing code snippets, or even the target audience for the output.

Context helps the AI generate responses that are consistent with your overall goals and existing codebase. The term "context" is used in several ways by various AI assistants. It can refer to the entire prompt submitted to the assistant, just the background information part of the prompt that the AI needs to understand the broader picture of your request, or a history of messages in the current conversation.

Suppose you've already generated a database schema. When asking for PHP code to interact with it, you would provide the schema as context.

Code Listing 2-e: Context Prompt Example

Given the following SQLite database schema:

```
```sql
CREATE TABLE documents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
type TEXT NOT NULL,
expiration_date TEXT NOT NULL,
notes TEXT
);
```
```

Write PHP code to connect to this SQLite database and insert a new document record.

The function should accept document name, type, expiration date, and notes as parameters.

By including the **CREATE TABLE** statement, you provide the AI with the necessary context about your database structure, enabling it to generate correct, compatible PHP code.

Role-playing and persona prompts

One powerful technique to guide the AI's output is to assign it a role or persona. By instructing the AI to act as a specific type of expert, you can influence its tone, style, and the kind of information it provides.

This is particularly useful in development, where different roles require different perspectives. Let's have a look at an example.

Code Listing 2-f: Persona Prompt

Act as a senior PHP developer with expertise in secure coding practices. Review the following PHP code for potential security vulnerabilities, including SQL injection and cross-site scripting (XSS).

For each vulnerability you find, explain the risk and provide a corrected version of the code:

[Paste your PHP code here]

By assigning the persona of a security-conscious senior developer, you prompt the AI to go beyond a simple code review and focus on a specific, critical aspect of software development, yielding much more valuable insights than a generic "review this code" prompt.

Iterative prompt refinement

A crucial point to understand is that it is rare to get the perfect response from an AI on the first try.

Iterative prompt refinement is the process of starting with a simple prompt and progressively improving it based on the AI's output. This feedback loop is a core part of practical prompt engineering.

The iterative process typically looks like this:

- **Submit initial prompt:** Start with a clear, specific, but not overly complex prompt.
- **Analyze output:** Review the AI's response. Does it meet your requirements? Is it accurate? Is anything missing?
- **Identify shortcomings:** Pinpoint where the output falls short. Perhaps it used the wrong library, misunderstood a requirement, or the code is inefficient.
- **Refine prompt:** Address the shortcomings. You might add more context, be more specific, provide an example, or correct a misunderstanding.
- **Repeat:** Continue this cycle until you achieve the desired output.

To get a better sense of this, let's look at an example.

Iteration 1: Initial prompt

Again, it is rare to get the perfect response from an AI on the first attempt.

Code Listing 2-g: Iteration 1 Prompt

```
Write a PHP function to get all documents from the database.
```

Now, let's do this for real and run that through the free version of [ChatGPT](#) to see what we get.

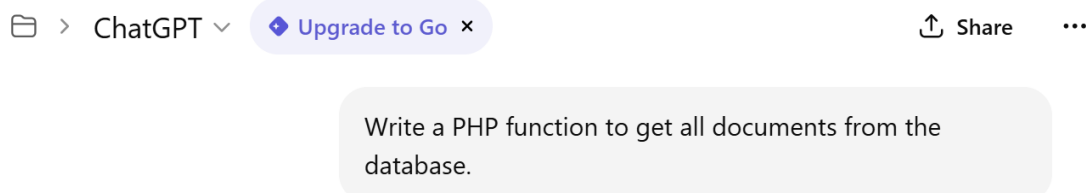


Figure 2-a: Iteration 1 (Using ChatGPT)

We should get back a reusable PHP function to fetch all documents from a database using PDO (which is safer and more modern than MySQLi).

For example, this is the code result I got from ChatGPT. Please note that you might not get the same result as I did.

Code Listing 2-h: Iteration 1 Resultant Code (ChatGPT Response)

```
<?php
function getAllDocuments($pdo) {
    try {
        $stmt = $pdo->prepare("SELECT * FROM documents");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        error_log("Database error: " . $e->getMessage());
        return [];
    }
}
```

Here's an example of how to use and invoke this code (also generated by ChatGPT).

Code Listing 2-i: How to Run the Resultant Code—Iteration 1 (ChatGPT Response)

```
<?php
// Database connection
$dsn = "mysql:host=localhost;dbname=my_database;charset=utf8mb4";
$user = "db_user";
$pass = "db_password";

try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
    ]);

    $documents = getAllDocuments($pdo);

    foreach ($documents as $doc) {
        echo $doc['title'] . "<br>";
    }
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

So, this is pretty good. We got the resultant code and how to implement it all in one shot.

Iteration 2: Refined prompt

So, the AI generated the function, but it returns a raw database result set that isn't very useful for display in an HTML template. Let's refine this.

Code Listing 2-j: Iteration 2 Prompt

Refine the previous function.

Instead of returning the raw database result, it should return an array of associative arrays, where each associative array represents a document with its properties (id, name, type, etc.)

📁 > ChatGPT ▾

📌 Upgrade to Go ×

🔗 Share

⋮

Refine the previous function.

Instead of returning the raw database result, it should return an array of associative arrays, where each associative array represents a document with its properties (id, name, type, etc.)

Figure 2-b: Iteration 2 (Using ChatGPT)

Here's a refined version of the function—it converts the raw result into a structured array of associative arrays and ensures each document has clearly named properties (**id**, **name**, **type**, etc.).

Code Listing 2-k: Iteration 2 Resultant Code (ChatGPT Response)

```
<?php
function getAllDocuments(PDO $pdo): array {
    try {
        $stmt = $pdo->prepare("SELECT id, name, type,
            created_at, updated_at FROM documents");
        $stmt->execute();
        $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

        $documents = [];
        foreach ($rows as $row) {
            $documents[] = [
```

```

        'id'          => (int)$row['id'],
        'name'       => $row['name'],
        'type'       => $row['type'],
        'created_at' => $row['created_at'],
        'updated_at' => $row['updated_at']
    ];
}

return $documents;
} catch (PDOException $e) {
    error_log("Database error in getAllDocuments: " .
        $e->getMessage());
    return [];
}
}

```

This code explicitly selects key fields (avoids `SELECT *`); converts IDs to integers for type consistency; wraps all data in a clear, structured array of associative arrays; logs errors; and returns an empty array on failure.

Now, let's look at how to invoke the code (also generated by ChatGPT).

Code Listing 2-1: How to Run the Resultant Code—Iteration 2 (ChatGPT Response)

```

<?php
$dsn = "mysql:host=localhost;dbname=my_database;charset=utf8mb4";
$user = "db_user";
$pass = "db_password";

try {
    $pdo = new PDO($dsn, $user, $pass,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
    $documents = getAllDocuments($pdo);

    foreach ($documents as $doc) {
        echo "{$doc['id']}: {$doc['name']} ({$doc['type']})<br>";
    }
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

```

So, are you seeing the pattern here? Each prompt iteration improves the resulting code based on the direction we want to go.

Another thing to note is that the LLM has memory, so during prompt refinement, there's no need to respecify the original prompt. Therefore, every prompt improvement is incremental.

Iteration 3: Further refined prompt

Let's take this a step further and see how we can add some more advanced error handling to the function. Let's use the following refined prompt.

Code Listing 2-m: Iteration 3 Prompt

Excellent. Now, add error handling to the function. If the database query fails, it should log the error to a file and return an empty array.

📁 > ChatGPT ▾ [Upgrade to Go](#) × [Share](#) ⋮

Excellent. Now, add error handling to the function. If the database query fails, it should log the error to a file and return an empty array.

Figure 2-c: Iteration 3 (Using ChatGPT)

After executing this prompt, ChatGPT returns an enhanced version of the function with robust error handling and file-based logging.

Code Listing 2-n: Iteration 3 Resultant Code (ChatGPT Response)

```
<?php
function getAllDocuments(PDO $pdo): array {
    $logFile = __DIR__ . '/error_log.txt'; // Log file path

    try {
        $stmt = $pdo->prepare("SELECT id, name, type,
            created_at, updated_at FROM documents");
        $stmt->execute();
        $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

        $documents = [];
        foreach ($rows as $row) {
            $documents[] = [
                'id'          => (int)$row['id'],
                'name'        => $row['name'],
```

```

        'type'      => $row['type'],
        'created_at' => $row['created_at'],
        'updated_at' => $row['updated_at']
    ];
}

return $documents;
} catch (PDOException $e) {
    // Log error with timestamp
    $errorMessage = sprintf("[%s] Database error: %s\n",
        date('Y-m-d H:i:s'), $e->getMessage());
    file_put_contents($logFile, $errorMessage, FILE_APPEND);

    // Optionally log stack trace for debugging
    // file_put_contents($logFile, $e->getTraceAsString() .
    // "\n", FILE_APPEND);

    return []; // Return empty array on failure
}
}

```

We can see the following improvements:

- **Error logging:** The generated PHP script writes detailed error messages to `error_log.txt` in the same directory as the script.
- **Time-stamped logs:** These make it easier to trace when issues occurred.
- **Safe fallback:** The script always returns an empty array in the event of a database failure.

Now, let's have a look at the example usage also generated by ChatGPT.

Code Listing 2-o: How to Run the Resultant Code—Iteration 3 (ChatGPT Response)

```

<?php
$dsn = "mysql:host=localhost;dbname=my_database;charset=utf8mb4";
$user = "db_user";
$pass = "db_password";

try {
    $pdo = new PDO($dsn, $user, $pass,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
    $documents = getAllDocuments($pdo);

    if (empty($documents)) {
        echo "No documents found or an error occurred.";
    }
}

```

```

    } else {
        foreach ($documents as $doc) {
            echo "{$doc['id']}: {$doc['name']} ({$doc['type']})<br>";
        }
    }
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

```

This is another excellent example of how to enhance a prompt by providing additional context to the AI assistant. Through this iterative process, we have guided the AI to produce a robust, well-structured, error-handling function ready for use in your application.

Output format specification

The last point we'll touch on before closing this chapter is how to ask AI to output results in a specific format. For any developer, specifying the desired output format can save significant time and effort in parsing and reformatting.

Standard output formats are:

- **Code blocks:** For code snippets, always ask the AI to format them in a code block with the appropriate language identifier (e.g., ``php``, ``sql``, ``html``).
- **JSON:** When you need structured data that your application can easily parse, JSON is an excellent choice.
- **Markdown:** A lightweight, readable format for documentation, explanations, or reports.
- **Tables:** When comparing options or presenting data, ask the AI to format the output as a table.

To illustrate this, let's go over an example prompt. Enter this prompt in the same ChatGPT window you've been using (do not start a new chat).

Code Listing 2-p: Specifying Output Format Prompt

I am building a REST API for my document tracker application. I need to define the API endpoints. Provide a list of endpoints in a Markdown table with the following columns:

```

'HTTP Method',
'Endpoint URL',
'Description',
and
'Example Request Body (JSON)'.

```

After executing this prompt, we get the following result from ChatGPT.

Here's a clean and practical list of **REST API endpoints** for your **Document Tracker** application — formatted as a Markdown table:

| HTTP Method | Endpoint URL | Description | Example Request Body (JSON) |
|-------------|---------------------|---|--|
| GET | /api/documents | Retrieve all documents. | N/A |
| GET | /api/documents/{id} | Retrieve a specific document by its ID. | N/A |
| POST | /api/documents | Create a new document. | json { "name": "Project Plan", "type": "PDF", "owner": "Alice Johnson" } |
| PUT | /api/documents/{id} | Update an existing | json { "name": "Project |

Figure 2-d: ChatGPT Result

Following is the result returned by ChatGPT, copied and pasted, formatted as a table.

Table 2-a

| HTTP Method | Endpoint URL | Description | Example Request Body (JSON) |
|-------------|---------------------|---|---|
| GET | /api/documents | Retrieve all documents. | N/A |
| GET | /api/documents/{id} | Retrieve a specific document by its ID. | N/A |
| POST | /api/documents | Create a new document. | json { "name": "Project Plan", "type": "PDF", "owner": "Alice Johnson" } |
| PUT | /api/documents/{id} | Update an existing document by ID. | json { "name": "Project Plan v2", "type": "PDF", "owner": "Alice Johnson" } |
| PATCH | /api/documents/{id} | Partially update specific fields of a document. | json { "status": "approved" } |

| HTTP Method | Endpoint URL | Description | Example Request Body (JSON) |
|-------------|---|--|---|
| DELETE | /api/documents/{id} | Delete a document by ID. | N/A |
| GET | /api/documents/search | Search documents by query or filters. | N/A (e.g., /api/documents/search?name=report&type=pdf) |
| GET | /api/documents/{id}/history | Retrieve version history or activity logs for a document. | N/A |
| POST | /api/documents/{id}/upload | Upload a file related to a document (metadata already exists). | multipart/form-data – file upload |
| GET | /api/status | Check API health or service status. | N/A |

Notice that ChatGPT did quite a good job because:

- All the endpoints are prefixed with [/api/](#) for clarity and versioning readiness (e.g., [/api/v1/documents](#) later).
- The API uses PATCH for partial updates instead of PUT when not all fields are supplied.
- For uploads, it uses [multipart/form-data](#) instead of JSON.
- You can expand with authentication routes like [/api/auth/login](#) and [/api/auth/register](#) if your tracker includes user management.

The prompt clearly specified the desired output format (a table) and the exact columns to include. This resulted in a well-structured, immediately usable response.

By mastering these fundamental prompting techniques, you will be well-equipped to tackle the more advanced concepts and practical application development in the upcoming chapters.

Chapter 3 Advanced Prompting

Overview

Building upon the foundational principles of clarity, specificity, and context, this chapter introduces more advanced prompting techniques that can unlock even greater capabilities in your AI assistant. These methods are beneficial for completing complex tasks, improving accuracy, and integrating AI seamlessly into your development workflow.

Few-shot prompting

Few-shot prompting is a powerful technique where you provide the AI with a few examples of input-output pairs before asking it to complete a new task. The term one-shot prompting is sometimes used to indicate a single example.

This helps the model understand the desired pattern, format, or behavior without requiring extensive fine-tuning. It's like showing a new intern a few completed tasks so they can replicate the style and quality.

How few-shot prompting works

By presenting a small number of examples, you implicitly teach the AI the nuances of the task. The model learns from these examples and applies that understanding to the new, unseen input. This is especially effective when the task is specific or requires a particular style that might not be obvious from a simple instruction.

Suppose you want the AI to generate PHP class structures for different entities in a consistent style. The prompt might be something like the following.

Code Listing 3-a: Few-Shot Prompting Example

```
Act as a senior PHP developer. I need you to generate PHP class definitions for various entities.
```

```
Follow the example provided for the types, the constructor, and the getter methods.
```

```
Example:
```

```
Input:
```

```
User with properties id (int), name (string), email (string)
```

Output:

Task:

Input: Document with properties id (int), expirationDate (string), notes (string, nullable)

Output:

After running this prompt, we get the following result from ChatGPT.

Code Listing 3-b: Few-Shot Prompting Result

```
<?php

class Document
{
    private int $id;
    private string $expirationDate;
    private ?string $notes;

    public function __construct(int $id, string $expirationDate,
        ?string $notes = null)
    {
        $this->id = $id;
        $this->expirationDate = $expirationDate;
        $this->notes = $notes;
    }

    public function getId(): int
    {
        return $this->id;
    }

    public function getExpirationDate(): string
    {
        return $this->expirationDate;
    }

    public function getNotes(): ?string
    {
        return $this->notes;
    }
}
```

Notice that this result includes:

- Typed properties (`int`, `string`, `?string`).
- A default value of `null` for the optional `notes` field in the constructor.
- A corresponding getter method for each property.
- A clean, modern PHP 8+ syntax.

By using this simple approach, the AI learns what the user expects as output. Now, let's explore another technique that takes this approach to the next level.

Chain-of-thought prompting

Chain-of-thought (CoT) prompting is a technique that encourages the AI to articulate its reasoning process step-by-step before arriving at a final answer. This approach is convenient for complex problems that require logical deduction, multistep reasoning, or breaking down an enormous task into smaller, manageable subtasks.

Why CoT prompting is effective

By forcing the AI to show its work, CoT prompting often leads to more accurate results and makes the AI's decision-making process transparent. It helps identify potential errors in reasoning and makes debugging the AI's output easier. It mimics how a human would approach a complex problem.

Suppose you have a PHP function that isn't working as expected and want the AI to help debug it.

Code Listing 3-c: CoT Example

Act as an experienced PHP debugger.

I have a PHP function that is supposed to calculate the number of days until a document expires.

It's not returning the correct value for some dates.

Please analyze the code step by step, identify the potential issue, explain your reasoning, and then provide the corrected code. Think step-by-step.

```
<?php
function daysUntilExpiration(string $expirationDate): int
{
    $today = new DateTime();
```

```

    $expiry = new DateTime($expirationDate);
    $interval = $today->diff($expiry);
    return $interval->days;
}

// Test cases
echo daysUntilExpiration('2025-10-10'); // Should be 3
echo daysUntilExpiration('2025-10-07'); // Should be 0
echo daysUntilExpiration('2025-10-06'); // Should be -1
?>

```

1. Analyze `daysUntilExpiration` function:

It uses `DateTime` objects and `diff()` to get the interval.

2. Examine `diff()` behavior:

The `diff()` method returns a `DateInterval` object. The `days` property of `DateInterval` always returns the absolute number of days. It does not indicate if the date is in the past or future.

3. Identify the issue:

For past dates, `diff()->days` will still be a positive number, which is incorrect for calculating days until expiration. We need to check the `invert` property of the `DateInterval` object to determine if the expiration date is in the past.

4. Formulate correction:

If `interval->invert` is 1, the expiration date is in the past, so the result should be negative. Otherwise, it's the positive number of days.

After running this prompt, we get back the following result from ChatGPT.

Code Listing 3-d: CoT Example Result

Good—let's debug this step by step and fix it.

1) Step-by-step analysis

- Your function builds two DateTime objects: \$today = new DateTime(); and \$expiry = new DateTime(\$expirationDate);.
- You call \$today->diff(\$expiry), which returns a DateInterval.
- You return \$interval->days.

Why this is wrong for "days until expiration":

- DateInterval::\$days is the total number of days between the two dates as an absolute value (it's always non-negative). It will not tell you whether the expiry is in the past or the future.

- `DateInterval` has an `invert` property: 0 means `$expiry >= $today` (future or same day), 1 means `$expiry < $today` (expiry in the past). You must use the inverse to sign the result.
- Another subtle bug: using `new DateTime()` includes the current time (hours/minutes/seconds).
- If `$expiry` is created as `2025-10-07 00:00:00` but `$today` is `2025-10-07 14:00:00`, `diff()` may count days differently, producing unexpected off-by-one results. For "days until expiration," you usually want to compare dates only (normalize to midnight).

2) Fix strategy

1. Normalize both dates to midnight so we compare date boundaries, not time of day.
2. Use the days value and apply the sign using the `invert` flag.
3. Return an `int` (positive for future, 0 for today, negative for past).

3) Corrected code

```
<?php
/**
 * Returns the number of days until $expirationDate.
 * Positive -> days in the future
 * 0 -> today
 * Negative -> days in the past
 *
 * @param string $expirationDate Date in 'YYYY-MM-DD'
 * (or any format accepted by DateTime)
 * @return int
 */
function daysUntilExpiration(string $expirationDate): int
{
    // Use DateTimeImmutable to avoid accidental mutation
    // (optional but good practice)
    $today = new DateTimeImmutable('today');
    // normalized to local timezone midnight
    try {
        $expiry = new DateTimeImmutable($expirationDate);
    } catch (Exception $e) {
        // If parse fails, log or rethrow – here we return 0 to indicate
        // "invalid" in a simple way.
        error_log("Invalid expiration date provided: {$expirationDate}");
        return 0;
    }

    // Normalize expiry to midnight too (in case time part was provided)
```

```

$expiry = $expiry->setTime(0, 0, 0);

$interval = $today->diff($expiry);

$days = (int)$interval->days;

// If invert == 1, expiry is in the past, so return negative days
return $interval->invert ? -$days : $days;
}

// Example test (assuming "today" is 2025-10-07 for these expectations):
echo daysUntilExpiration('2025-10-10') . PHP_EOL; // => 3
echo daysUntilExpiration('2025-10-07') . PHP_EOL; // => 0
echo daysUntilExpiration('2025-10-06') . PHP_EOL; // => -1

```

4) Notes & recommendations

- If you need to force a specific timezone, construct `DateTimeImmutable` with a `DateTimeZone` object:
`new DateTimeImmutable('today', new DateTimeZone('Europe/Madrid'))`.
- If you want stricter handling of invalid input, throw an exception instead of returning 0.
- Using `DateTimeImmutable` avoids accidental side-effects; you can switch to `DateTime` if you prefer.

That fixes both the sign problem (use `invert`) and the possible time-of-day off-by-one (normalize to midnight).

By explicitly asking the AI to "think step-by-step," you guide it to perform a more thorough analysis, which is invaluable for debugging and understanding complex logic.

Self-correction and debugging with AI

AI models can be powerful allies in the debugging process, not just by identifying errors but also by suggesting self-correction mechanisms. This involves using the AI to analyze its own previous outputs or to review code for potential issues and then propose fixes. This technique leverages the iterative nature of prompt engineering to improve the quality of AI-generated code.

Instead of just asking for code, you can ask the AI to act as a code reviewer, identifying common pitfalls, suggesting improvements, or even fixing bugs in code you provide (whether AI-generated or human-written). Let's look at an example.

Code Listing 3-e: Self-Correction/Code Review Example

Act as a meticulous PHP code reviewer. I have the following PHP function that handles user input.

Please review it for potential security vulnerabilities (e.g., SQL injection and XSS), common coding errors, and adherence to best practices.

If you find any issues, explain them and provide a corrected version of the function.

```
<?php

function processUserInput($input)
{
    $name = $_POST["name"];
    $email = $_POST["email"];
    $age = $_POST["age"];
    // Assume $db is a global SQLite PDO connection.
    global $db;

    $stmt = $db->prepare("INSERT INTO users (name, email, age) VALUES
    (\".$name.\", \".$email.\", \".$age.\")");
    $stmt->execute();
    echo "User added successfully!";
}
```

The AI should identify the direct concatenation of user input into the SQL query as a severe SQL injection vulnerability. It would then explain the risk and provide a corrected version using prepared statements with parameter binding.

Now, after executing this prompt, we get the following result from ChatGPT.

Code Listing 3-f: Self-Correction/Code Review Example Result

1. SQL Injection vulnerability

The following line is dangerous:

```
$stmt = $db->prepare("INSERT INTO users (name, email, age) VALUES
(\".$name.\", \".$email.\", \".$age.\")");
```

- The user-provided variables \$name, \$email, and \$age are concatenated directly into the SQL query string.

- This completely bypasses PDO's protection, making the query vulnerable to SQL injection and allowing attackers to manipulate the Database (e.g., deleting tables or extracting data).

Fix: Always use parameterized queries with placeholders and bind values safely.

2. Cross-Site Scripting (XSS)

The line:

```
echo "User added successfully!";
```

It is not itself dangerous, but if you ever echo user-supplied input back to the browser (like `$name` or `$email`), it should be escaped with `htmlspecialchars()` to prevent XSS.

Fix: Use `htmlspecialchars()` when outputting user data in HTML.

3. Input validation

There's no validation or sanitization of the incoming data:

```
$name = $_POST["name"];
```

```
$email = $_POST["email"];
```

```
$age = $_POST["age"];
```

- `$age` should be numeric.
- `$email` should be validated with `filter_var($email, FILTER_VALIDATE_EMAIL)`.
- `$name` should be trimmed and restricted to a reasonable length/characters.

Fix: Validate and sanitize each field before inserting.

4. Error handling

No error handling is present. If the query fails, the user still sees "User added successfully!", which is misleading.

Fix: Wrap the database call in a try/catch block and provide a safe fallback message.

5. Global variable use

Using global `$db`; is not ideal—it makes the function harder to test and maintain.

Fix: Pass the database connection as a parameter.

Corrected version

Here's a secure, maintainable rewrite:

```
<?php
function processUserInput(PDO $db): void
{
    // Validate and sanitize input
    $name = trim($_POST['name'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $age = $_POST['age'] ?? null;

    if (empty($name) || !filter_var($email, FILTER_VALIDATE_EMAIL) ||
        !is_numeric($age)) {
        echo "Invalid input. Please check your data.";
        return;
    }

    try {
        $stmt = $db->prepare("
            INSERT INTO users (name, email, age)
            VALUES (:name, :email, :age)
        ");
        $stmt->bindParam(':name', $name, PDO::PARAM_STR);
        $stmt->bindParam(':email', $email, PDO::PARAM_STR);
        $stmt->bindParam(':age', $age, PDO::PARAM_INT);
        $stmt->execute();

        echo "User added successfully!";
    } catch (PDOException $e) {
        // Log error to file, not shown to user
        error_log("Database error in processUserInput: " .
            $e->getMessage());
        echo "An error occurred while adding the user.";
    }
}
```

This demonstrates how AI can act as a proactive security and quality-assurance assistant, catching errors that might otherwise go unnoticed.

Integrating AI into your workflow

The ultimate goal of mastering advanced prompting techniques is to integrate AI seamlessly into your daily development workflow, making it an extension of your own capabilities.

This isn't about replacing human developers but about augmenting them, enabling greater efficiency, creativity, and focus on higher-level problems.

Practical integration strategies

- **IDE extensions:** Many AI assistants offer IDE integrations (such as GitHub Copilot for VS Code) that provide real-time code suggestions, autocompletion, and even the generation of entire functions as you type. This is the most direct form of integration.
- **Dedicated AI chat interfaces:** Use web-based or desktop AI chat applications for more complex tasks that require multiturn conversations, detailed explanations, or brainstorming sessions. Copying and pasting code snippets and prompts is common here.
- **Command-line tools and APIs:** For automated tasks or integrating AI into scripts, leverage AI APIs. You can write scripts that send prompts to an LLM and process its responses, automating tasks such as documentation generation, code analysis, and even test-case creation.
- **Version control integration:** Integrate AI into your Git workflow. For example, an AI could generate commit messages based on code changes or review pull requests for potential issues.

Best practices for workflow integration

- **Start small:** Begin by integrating AI for simple, repetitive tasks. As you gain confidence, gradually expand its role to more complex problems.
- **Verify everything:** Always review AI-generated code and suggestions. Treat AI output as a starting point, not a final solution. Human oversight is critical.
- **Maintain context:** When switching between tasks or tools, ensure the AI has the necessary context to provide relevant responses. This might involve copying previous conversation turns or relevant code snippets.
- **Learn from AI:** Pay attention to the patterns and solutions the AI provides. This can help you learn new techniques, discover alternative approaches, and improve your own coding skills.
- **Document your prompts:** For complex or frequently used tasks, document the effective prompts you've developed. This creates a reusable knowledge base and ensures consistency.

By strategically integrating AI into your development workflow and continuously refining your prompting skills, you can significantly boost your productivity and focus on the more challenging and creative aspects of software development.

The next part of the book will put these principles into practice as we begin building our document-expiration-tracking solution.

Chapter 4 Kick-Off: Database Design

Overview

With a solid understanding of prompt engineering fundamentals and advanced techniques, it's time to apply this knowledge to building our document expiration tracker application. We won't have time to make a full-blown app, but we'll focus on its core features.

This chapter marks the project kick-off, focusing on the crucial first step: designing and implementing the database. We will use ChatGPT, our AI assistant, to guide us through understanding the application requirements and generating the SQLite database schema.

Understanding the application requirements

Before diving into code, it's essential to define what our application needs to do clearly. This clarity will be paramount when crafting effective prompts for the AI. Our document expiration tracker needs to:

- **Store document information:** Each document (e.g., driver's license, passport, contract) must have a name, type, expiration date, and optional notes.
- **Identify documents uniquely:** Each document needs a unique identifier.
- **Track expiration dates:** The primary function is to monitor and display documents based on their expiration dates.
- **Basic management:** Users should be able to add, view, edit, and delete document records.

This simple set of requirements forms the basis for our database design. We will use SQLite for its simplicity and file-based nature, which make it ideal for a small, self-contained application like this.

Database design principles

Before we ask the AI to generate our schema, let's briefly review some fundamental database design principles that guide our requirements:

- **Normalization:** Aim to reduce data redundancy and improve data integrity. For our simple application, a single table is sufficient; for more complex systems, data would be split across multiple related tables.
- **Primary keys:** Every table should have a primary key, which is a column (or set of columns) that uniquely identifies each record. This is crucial for efficient data retrieval and relationships.

- **Data types:** Choosing appropriate data types for each column ensures data integrity and optimizes storage and performance. SQLite is flexible with data types, but it's good practice to define them clearly.
- **Constraints:** These ensure that critical fields are never left empty, maintaining data quality.

Database design principles

SQLite uses a more dynamic type system than other relational databases. It has five primary storage classes:

- **NULL:** The value is a NULL value.
- **INTEGER:** The value is a signed integer, stored in 1, 2, 3, 4, 5, 6, or 8 bytes depending on the magnitude of the value.
- **REAL:** The value is a floating point value, stored as an 8-byte IEEE floating point number.
- **TEXT:** The value is a text string, stored using the database encoding (UTF-8, UTF-16BE, or UTF-16LE).
- **BLOB:** The value is a blob of data, stored exactly as it was input.

When you declare a type like **VARCHAR(255)** or **DATETIME**, SQLite uses a concept called type affinity to determine the storage class. For example, **DATETIME** columns typically have **NUMERIC** or **TEXT** affinity. For dates, storing them as **TEXT** in 'YYYY-MM-DD' format is often the most straightforward and compatible approach for sorting and comparison in SQLite without complex conversions.

Let's use our AI assistant to help us design the SQLite database schema based on the requirements and these principles. We'll employ the principles of clarity, specificity, and output format specification to get a precise statement.

Code Listing 4-a: DB Schema Prompt

Act as a database administrator specializing in SQLite.

I am building a PHP application to track documents with expiration dates. Based on the following requirements, generate a `CREATE TABLE` statement for an SQLite database.

The table should be named `documents`.

Requirements:

- Each document needs a unique identifier.
- Store the document's name (e.g., "Driver's License"). This should be a required text field.
- Store the document's type (e.g., "License", "Passport", "Contract").

- This should also be a required text field.
- Store the expiration date. This is a critical field and should be a required text field, storing dates in 'YYYY-MM-DD' format for easy comparison.
- Allow for optional notes about the document. This can be a text field.

Ensure the primary key is auto-incrementing. Provide only the SQL `CREATE TABLE`` statement, enclosed in a SQL code block.

After entering this prompt, we get the following result from ChatGPT.

Code Listing 4-b: DB Schema Prompt Result

```
CREATE TABLE documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  type TEXT NOT NULL,
  expiration_date TEXT NOT NULL CHECK(length(expiration_date) = 10),
  notes TEXT
);
```

The AI, acting as a SQLite database administrator, has provided a perfectly valid statement. Let's break down its components:

- **id INTEGER PRIMARY KEY AUTOINCREMENT:** This defines a unique identifier for each document. **INTEGER PRIMARY KEY** in SQLite automatically makes the column auto-incrementing, ensuring each new record gets a unique ID.
- **name TEXT NOT NULL:** Stores the document's name as text, and **NOT NULL** ensures that this field cannot be empty.
- **type TEXT NOT NULL:** Stores the document's type as text, also a required field.
- **expiration_date TEXT NOT NULL:** Stores the expiration date as text. While SQLite supports various date and time functions, storing dates as 'YYYY-MM-DD' strings is a common and efficient approach for sorting and comparison without complex type conversions.
- **notes TEXT:** An optional text field for any additional information.

This schema directly addresses all our initial requirements, demonstrating that a well-crafted prompt can yield precise, usable database definitions.

Creating the database and tables

Now that we have our **CREATE TABLE** statement, the next step is to actually create the SQLite database file and the documents table within it.

We can use PHP's PHP Data Objects (PDO) extension to interact with SQLite.

Code Listing 4-c: Creating Database and Tables Prompt

Act as a PHP developer.

I need a PHP script that will create an SQLite database file named `documents.sqlite` in the same directory as the script, and then execute the following `CREATE TABLE` statement to create the `documents` table.

The script should handle potential errors during database connection or table creation.

If successful, it should print "Database and table created successfully!", and if not, it should print an error message.

Provide the complete PHP script, enclosed in a PHP code block.

```
```sql
CREATE TABLE documents (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 type TEXT NOT NULL,
 expiration_date TEXT NOT NULL CHECK(length(expiration_date) = 10),
 notes TEXT
);
```

After entering this prompt, we get the following result from ChatGPT.

*Code Listing 4-d: Creating Database and Tables Prompt Result (create\_documents\_db.php)*

```
<?php
// Filename: create_documents_db.php

$dbFile = __DIR__ . '/documents.sqlite';

try {
 // Create (or open) the SQLite database
 $db = new PDO('sqlite:' . $dbFile);
 $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

 // SQL statement to create the documents table
 $createTableSQL = "
 CREATE TABLE IF NOT EXISTS documents (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

 name TEXT NOT NULL,
 type TEXT NOT NULL,
 expiration_date TEXT NOT NULL CHECK(length(expiration_date) = 10),
 notes TEXT
);
";

// Execute the SQL command
$db->exec($createTableSQL);

echo "Database and table created successfully!";
} catch (PDOException $e) {
 echo "Error creating database or table: " .
 htmlspecialchars($e->getMessage());
}

```

This PHP script should handle the database creation. Let's go over what it does:

- `$dbFile = __DIR__ . '/documents.sqlite';` defines the path for our SQLite database file. `__DIR__` ensures it's created in the same directory as the script.
- `new PDO('sqlite:' . $dbFile);` This line attempts to connect to the SQLite database. If `documents.sqlite` doesn't exist, PDO will create it.
- `setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);` This is a crucial setting that configures PDO to throw exceptions on errors, allowing us to catch them with a `try-catch` block.
- `CREATE TABLE IF NOT EXISTS documents;` The `IF NOT EXISTS` clause is a good practice, preventing an error if the table already exists.
- `exec($createTableSQL);` Executes the SQL command to create the table.
- `try-catch (PDOException $e);` This block gracefully handles database errors, printing a user-friendly message and logging a detailed error message.

To run this script, save it as `create_db.php` in your web server's document root (e.g., `htdocs` for XAMPP) and access it via your browser (e.g., `http://localhost/create_db.php`). Note that in my case, the Apache web server is running on port 80.



Figure 4-a: XAMPP Control Panel (Apache Web Server—Port 80)

Then, open the browser to run the PHP script.

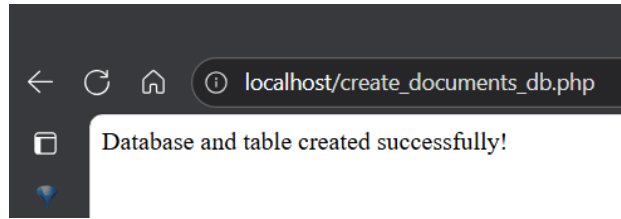


Figure 4-b: Successful Execution of the PHP Script

After execution, you should find the **documents.sqlite** file in the same directory.

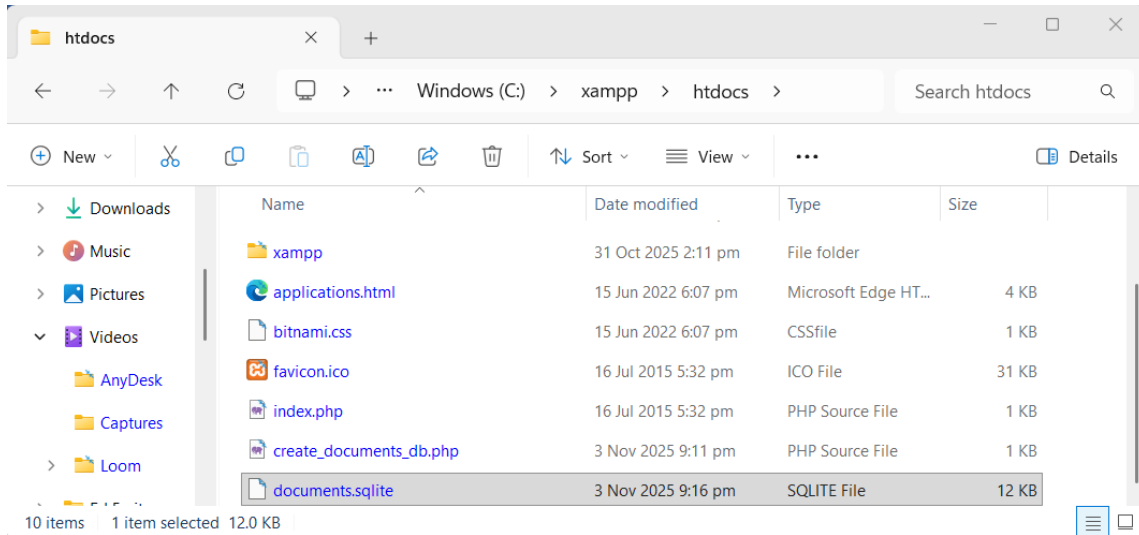


Figure 4-c: The documents.sqlite File Successfully Created

To view the content of the documents.sqlite file, install [DB Browser for SQLite](#), and then run the app. Once the app is running, go to the **File** menu, click **Open Database**, and select the file.

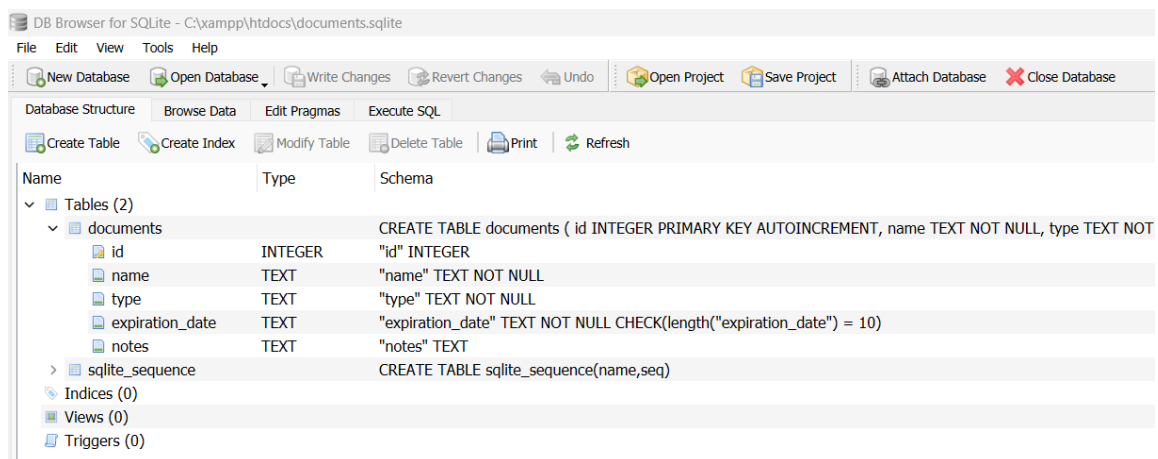


Figure 4-d: The documents.sqlite File—DB Browser for SQLite

With the DB Browser for SQLite app, you'll be able to see and manipulate the file structure and data.

## Initial data population prompts

To make our application functional and testable, it's helpful to populate the database with some initial data. We can again use our AI assistant, ChatGPT, to generate statements.

*Code Listing 4-e: Generate Data Prompt*

```
Act as a data entry specialist. Generate 5 `INSERT` statements for the
`documents` table, using the schema provided previously.

Include a mix of document types and expiration dates, some in the near
future, some further out, and at least one that is already expired (e.g.,
'2024-01-15').

Ensure the dates are in 'YYYY-MM-DD' format. Provide only the SQL
`INSERT` statements, each on a new line, enclosed in a SQL code block.
SQLite `documents` table schema:

```sql
CREATE TABLE documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  type TEXT NOT NULL,
  expiration_date TEXT NOT NULL CHECK(length(expiration_date) = 10),
  notes TEXT
);
```

After executing this prompt, we get the following result from ChatGPT.

Code Listing 4-f: Generate Data Prompt Result

```
INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Driver\'s License', 'License', '2024-01-15', 'Expired - needs
renewal');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Passport', 'Passport', '2027-06-30', 'Valid for international travel');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Employment Contract', 'Contract', '2026-12-31', 'Signed with HR');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Health Insurance Card', 'Insurance', '2025-05-20', 'Renew annually');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Building Lease Agreement', 'Contract', '2025-11-01', 'Office space
lease renewal due soon');
```

ChatGPT has generated a set of **INSERT** statements that populate our documents table with diverse data, including an expired document.

To execute these **INSERT** statements, we can create a new PHP script.

Code Listing 4-g: Prompt to Create PHP to Insert Data

```
Create a PHP script that inserts the following data into the recently created SQLite database.
```

```
INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Driver\'s License', 'License', '2024-01-15', 'Expired - needs
renewal');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Passport', 'Passport', '2027-06-30', 'Valid for international travel');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Employment Contract', 'Contract', '2026-12-31', 'Signed with HR');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Health Insurance Card', 'Insurance', '2025-05-20', 'Renew annually');

INSERT INTO documents (name, type, expiration_date, notes) VALUES
('Building Lease Agreement', 'Contract', '2025-11-01', 'Office space
lease renewal due soon');
```

Notice that we are not even mentioning the database or table name, because ChatGPT already knows the context from our previous prompts.

Code Listing 4-h: Create PHP to Insert Data Result (insert_documents_db.php)

```
<?php
// Filename: insert_documents.php

$dbFile = __DIR__ . '/documents.sqlite';

try {
    // Connect to the SQLite database
    $db = new PDO('sqlite:' . $dbFile);
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Array of INSERT statements
    $insertStatements = [
        "INSERT INTO documents (name, type, expiration_date, notes)
        VALUES ('Driver's License', 'License', '2024-01-15',
        'Expired - needs renewal');",
```

```

        "INSERT INTO documents (name, type, expiration_date, notes)
        VALUES ('Passport', 'Passport', '2027-06-30',
        'Valid for international travel');"
        "INSERT INTO documents (name, type, expiration_date, notes)
        VALUES ('Employment Contract', 'Contract', '2026-12-31',
        'Signed with HR');"
        "INSERT INTO documents (name, type, expiration_date, notes)
        VALUES ('Health Insurance Card', 'Insurance', '2025-05-20',
        'Renew annually');"
        "INSERT INTO documents (name, type, expiration_date, notes)
        VALUES ('Building Lease Agreement', 'Contract', '2025-11-01',
        'Office space lease renewal due soon');"
    ];

    // Begin transaction for efficiency and atomicity
    $db->beginTransaction();

    foreach ($insertStatements as $sql) {
        $db->exec($sql);
    }

    $db->commit();

    echo "Records inserted successfully!";
} catch (PDOException $e) {
    // Roll back on error
    if ($db->inTransaction()) {
        $db->rollBack();
    }
    echo "Error inserting records: " . htmlspecialchars(
        $e->getMessage());
}

```

We can save this file as **insert_documents_db.php** and then open the browser on the **localhost** to execute the script.

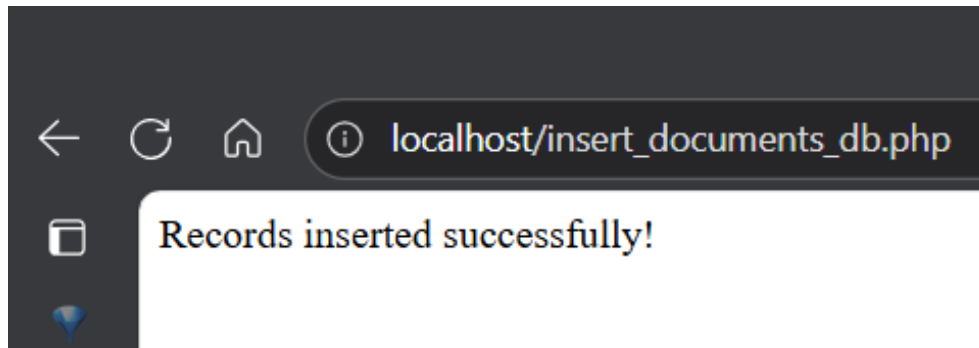


Figure 4-e: Successful Execution of the PHP Script (*insert_documents_db.php*)

If we open the **documents.sqlite** file, and then go to the **Browse Data** tab, we'll see the inserted data.

 A screenshot of the 'DB Browser for SQLite' application window. The title bar indicates the file path 'C:\xampp\htdocs\documents.sqlite'. The 'Browse Data' tab is selected, showing a table named 'documents'. The table has five columns: 'id', 'name', 'type', 'expiration_date', and 'notes'. There are five rows of data listed below the header.

| | id | name | type | expiration_date | notes |
|---|----|--------------------------|-----------|-----------------|-------------------------------------|
| 1 | 1 | Driver's License | License | 2024-01-15 | Expired - needs renewal |
| 2 | 2 | Passport | Passport | 2027-06-30 | Valid for international travel |
| 3 | 3 | Employment Contract | Contract | 2026-12-31 | Signed with HR |
| 4 | 4 | Health Insurance Card | Insurance | 2025-05-20 | Renew annually |
| 5 | 5 | Building Lease Agreement | Contract | 2025-11-01 | Office space lease renewal due soon |

Figure 4-f: The Inserted Data—DB Browser for SQLite

We are now ready to start with the core application logic.

Chapter 5 Core App: CRUD Operations

Overview

With our SQLite database schema defined and initial data populated, we can now turn our attention to the core functionality of the document expiration tracker: CRUD operations. CRUD stands for create, read, update, and delete, representing the four fundamental operations required to manage persistent data in any application.

In this chapter, we will systematically prompt our AI assistant to generate the PHP code necessary to perform these operations on our documents table.

Understanding CRUD operations

CRUD operations are the backbone of most data-driven applications. They allow users to interact with the data stored in a database.

For our document expiration tracker, the CRUD operations will do the following:

- **Create:** Add new document records (e.g., a new passport).
- **Read:** Display a list of all documents or view the details of a specific document.
- **Update:** Modify existing document records (e.g., updating an expiration date).
- **Delete:** Remove document records (e.g., when a document is no longer needed).

We will use PHP with PDO (PHP Data Objects) for database interaction, providing a lightweight, secure way to communicate with SQLite.

Prompting for PHP database connection

First, we need a reliable way to connect to our SQLite database. It's good practice to encapsulate this logic into a reusable function or class.

We'll ask the AI to provide a function that returns a PDO object.

Code Listing 5-a: Prompt for Database Connection

```
Act as a PHP developer specializing in database interactions.
```

```
I need a PHP function named `getDbConnection()` that establishes a  
connection to an SQLite database file called `documents.sqlite`.
```

```
The database file is located in the same directory as the PHP script.
```

The function should return a PDO object configured to throw exceptions on errors.

Include proper error handling. Provide only the complete PHP function, enclosed in a PHP code block.

Let's have a look at ChatGPT's AI-generated response.

Code Listing 5-b: Prompt for Database Connection Result (db_connection.php)

```
<?php
function getDbConnection() {
    try {
        // Path to the SQLite database file (
        // same directory as this script)
        $dbPath = __DIR__ . '/documents.sqlite';

        // Create a new PDO connection
        $pdo = new PDO("sqlite:" . $dbPath);

        // Set error mode to exceptions
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        return $pdo;
    } catch (PDOException $e) {
        // Handle connection errors
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}
```

If you place this file in the **htdocs** folder of your Apache local web server and run it in the browser, you should see a blank screen—don't panic. I'll explain why shortly.

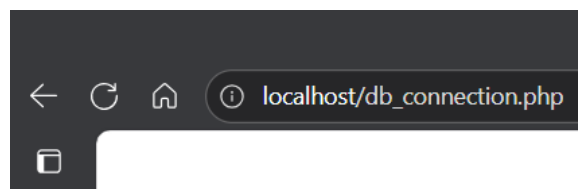


Figure 5-a: The Database Connection Result

This `getDbConnection()` function is robust and reusable:

- `$dbPath = __DIR__ . '/documents.sqlite'`: Dynamically determines the path to our database file.

- `$pdo = new PDO("sqlite:" . $dbPath);` Creates a new PDO instance, connecting to our SQLite database.
- `$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);` Crucially sets PDO to throw a PDOException on errors, allowing us to catch and handle them gracefully.
- **try-catch block:** Ensures that any connection errors are logged and the application exits cleanly, preventing sensitive database information from being exposed to the user.

We will include this function at the top of any PHP script that needs to interact with the database.

Generating code for listing documents (read)

The first core operation is to read and display all documents. This involves fetching data via a **SELECT** statement from the documents table and presenting it in a structured format.

We'll need two primary read functions: one to retrieve all documents for a list view, and another to retrieve a single document by its ID for editing.

Retrieving all documents

Let's write a prompt to retrieve all documents from the database and explicitly include the `getDbConnection()` function.

Code Listing 5-c: Get a List of All the Documents

Act as a PHP developer. Using the `getDbConnection()` function provided previously (**also INCLUDE IT**), write a PHP function named `getAllDocuments()` that retrieves all records from the `documents` table.

The function should return an array of associative arrays, where each inner associative array represents a document with its `id`, `name`, `type`, `expiration_date`, and `notes`.

Ensure proper error handling and use prepared statements if applicable (though for a simple `SELECT *` without `WHERE` clauses, it's less critical but good practice).

Order the results by `expiration_date` in ascending order.

Then, we get this result from ChatGPT.

Code Listing 5-d: Get a List of All the Documents Result (all_docs.php)

```
<?php
function getDbConnection() {
    try {
        // Path to the SQLite database file (
        // same directory as this script)
        $dbPath = __DIR__ . '/documents.sqlite';

        // Create a new PDO connection
        $pdo = new PDO("sqlite:" . $dbPath);

        // Set error mode to exceptions
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        return $pdo;
    } catch (PDOException $e) {
        // Handle connection errors
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

function getAllDocuments() {
    try {
        // Establish database connection
        $pdo = getDbConnection();

        // Prepare SQL statement
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            ORDER BY expiration_date ASC
        ");

        // Execute the query
        $stmt->execute();

        // Fetch all results as associative arrays
        $documents = $stmt->fetchAll(PDO::FETCH_ASSOC);

        return $documents;
    } catch (PDOException $e) {
        // Log and handle database errors
    }
}
```

```

        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

```

Let's place the generated file within the **htdocs** folder and open it in the browser. Notice how the browser returns nothing.

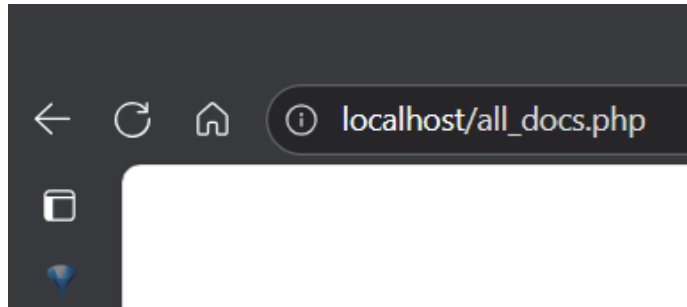


Figure 5-b: The all_docs.php Execution Result

So, what's going on? Why is nothing being returned? Well, why don't we ask ChatGPT?

Code Listing 5-e: Asking ChatGPT

I'm not getting anything when running this file

ChatGPT provides the following output.

Code Listing 5-f: Result After Asking ChatGPT

Your functions are correct syntactically, but nothing will show unless you actually call `getAllDocuments()` and output the results.

Here's how to fix that

```

<?php
function getDbConnection() {
    try {
        // Path to the SQLite database file (
        // same directory as this script)
        $dbPath = __DIR__ . '/documents.sqlite';

        // Create a new PDO connection
        $pdo = new PDO("sqlite:" . $dbPath);

        // Set error mode to exceptions
    }
}

```

```

        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        return $pdo;
    } catch (PDOException $e) {
        // Handle connection errors
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

function getAllDocuments() {
    try {
        // Establish database connection
        $pdo = getDbConnection();

        // Prepare SQL statement
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            ORDER BY expiration_date ASC
        ");

        // Execute the query
        $stmt->execute();

        // Fetch all results as associative arrays
        $documents = $stmt->fetchAll(PDO::FETCH_ASSOC);

        return $documents;
    } catch (PDOException $e) {
        // Log and handle database errors
        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

// Add this actually to fetch and display results
try {
    $documents = getAllDocuments();

    if (empty($documents)) {
        echo "No documents found in the database.";
    } else {
        echo "<pre>";
        print_r($documents);
    }
}

```

```

        echo "</pre>";
    }
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}

```

Check these common issues:

1. Database file location
 - Make sure documents.sqlite is in the same directory as this PHP file.
 - You can test with:
 - `var_dump(file_exists(__DIR__ . '/documents.sqlite'));`
 - It should output `bool(true)`.
2. Table name
 - Ensure the table is actually called documents (not Documents or docs).
 - You can verify it by opening SQLite CLI:
 - `sqlite3 documents.sqlite`
 - `.tables`
3. Data inserted
 - If you haven't run your INSERT script yet, run it once to populate data.
4. PHP error display
 - Enable error reporting at the top of the file:
 - `ini_set('display_errors', 1);`
 - `error_reporting(E_ALL);`

The interesting part is the portion I have highlighted in yellow. Essentially, we are not invoking the `getAllDocuments` function, so nothing is returned.

Now, the script should return the result in the browser, and the updated `all_docs.php` file should look like this.

Code Listing 5-g: Updated `all_docs.php` File

```

<?php
function getDbConnection() {
    try {
        // Path to the SQLite database file (
        // same directory as this script)
        $dbPath = __DIR__ . '/documents.sqlite';

        // Create a new PDO connection
        $pdo = new PDO("sqlite:" . $dbPath);

        // Set error mode to exceptions
    }
}

```

```

        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        return $pdo;
    } catch (PDOException $e) {
        // Handle connection errors
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

function getAllDocuments() {
    try {
        // Establish database connection
        $pdo = getDbConnection();

        // Prepare SQL statement
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            ORDER BY expiration_date ASC
        ");

        // Execute the query
        $stmt->execute();

        // Fetch all results as associative arrays
        $documents = $stmt->fetchAll(PDO::FETCH_ASSOC);

        return $documents;
    } catch (PDOException $e) {
        // Log and handle database errors
        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

// Add this actually to fetch and display results
try {
    $documents = getAllDocuments();

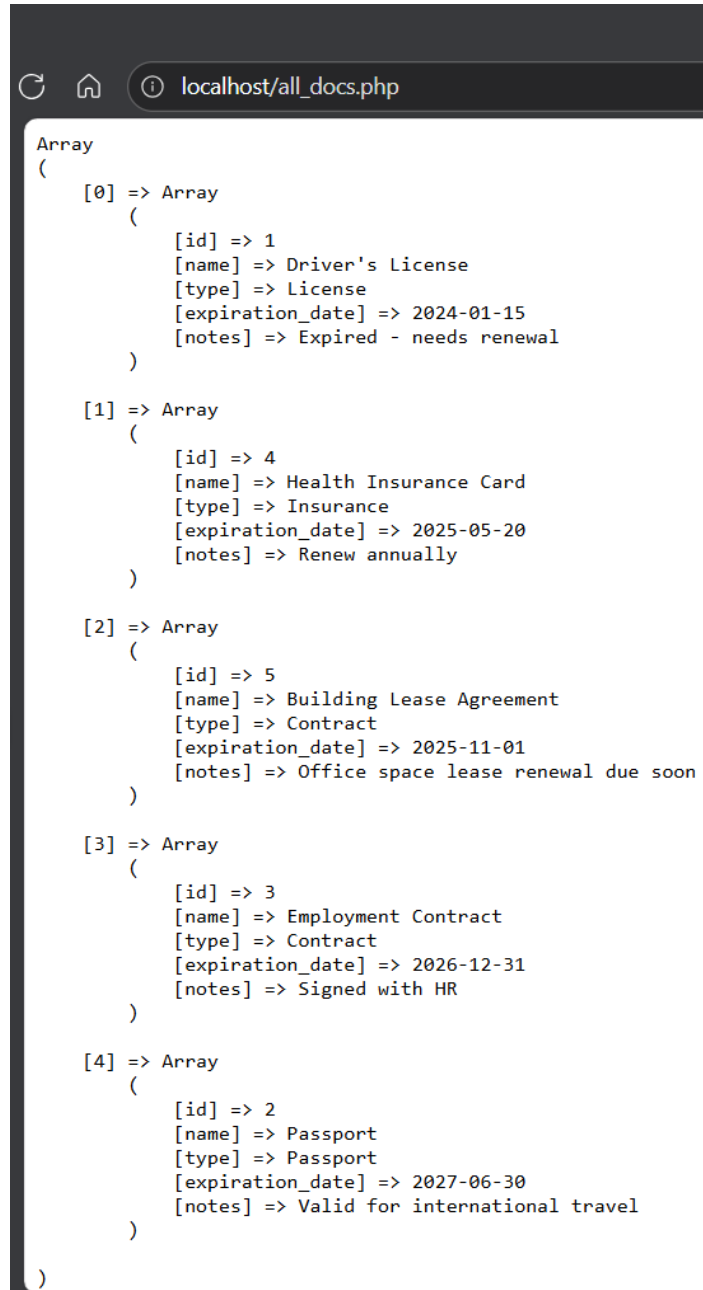
    if (empty($documents)) {
        echo "No documents found in the database.";
    } else {
        echo "<pre>";
    }
}

```

```

        print_r($documents);
        echo "</pre>";
    }
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}

```



```

Array
(
    [0] => Array
        (
            [id] => 1
            [name] => Driver's License
            [type] => License
            [expiration_date] => 2024-01-15
            [notes] => Expired - needs renewal
        )

    [1] => Array
        (
            [id] => 4
            [name] => Health Insurance Card
            [type] => Insurance
            [expiration_date] => 2025-05-20
            [notes] => Renew annually
        )

    [2] => Array
        (
            [id] => 5
            [name] => Building Lease Agreement
            [type] => Contract
            [expiration_date] => 2025-11-01
            [notes] => Office space lease renewal due soon
        )

    [3] => Array
        (
            [id] => 3
            [name] => Employment Contract
            [type] => Contract
            [expiration_date] => 2026-12-31
            [notes] => Signed with HR
        )

    [4] => Array
        (
            [id] => 2
            [name] => Passport
            [type] => Passport
            [expiration_date] => 2027-06-30
            [notes] => Valid for international travel
        )
)

```

Figure 5-c: The `all_docs.php` Successful Execution

Awesome. There are two takeaways from this exercise. The first is to request that ChatGPT include any previous, related functions, and the second is to indicate that those functions should be invoked.

Retrieving a single document by ID

For editing or viewing the details of a specific document, we need a function to fetch a single record based on its unique ID. Let's prompt the AI to create a consolidated file for CRUD operations.

Code Listing 5-h: First Prompt to Create a Consolidated CRUD File

Act as a PHP developer.

Using the ``getDbConnection()`` and ``getDocuments()`` functions, write a PHP function named ``getDocumentById(int $id): ?array`` that retrieves a single document record from the ``documents`` table based on its ``id``.

The function should return an associative array representing the document if found, or ``null`` if no document with the given ID exists.

Use prepared statements for secure querying and include proper error handling.

Consolidate the previous prompt resultant code along with this one.

Name this new file `crud.php`, and it should work for getting all documents and a single document.

Alright, let's see what result we get.

Code Listing 5-i: First Prompt to Create a Consolidated CRUD File Result (crud.php)

Here's your complete, production-ready `crud.php` file, containing all three functions:

- `getDbConnection()`
- `getAllDocuments()`
- `getDocumentById(int $id): ?array`

This script safely connects to an SQLite database (`documents.sqlite`) and either retrieves all documents or a single document by ID.

It includes error handling, prepared statements, and sample usage for both functions.

```

<?php
/**
 * crud.php
 * SQLite CRUD operations for the 'documents' table.
 */

ini_set('display_errors', 1);
error_reporting(E_ALL);

/**
 * Establishes a connection to the SQLite database.
 *
 * @return PDO
 * @throws Exception if the connection fails
 */
function getDbConnection(): PDO {
    try {
        $dbPath = __DIR__ . '/documents.sqlite';
        $pdo = new PDO("sqlite:" . $dbPath);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

/**
 * Retrieves all document records from the 'documents' table.
 *
 * @return array Returns an array of associative arrays (documents)
 * @throws Exception if the query fails
 */
function getAllDocuments(): array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            ORDER BY expiration_date ASC
        ");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {

```

```

        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

/**
 * Retrieves a single document by its ID.
 *
 * @param int $id The document ID
 * @return array|null Returns the doc as an array, or null if not found
 */
function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            WHERE id = :id
            LIMIT 1
        ");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        $document = $stmt->fetch(PDO::FETCH_ASSOC);
        return $document !== false ? $document : null;
    } catch (PDOException $e) {
        error_log("Error fetching document by ID: " . $e->getMessage());
        return null;
    }
}

/**
 * Example usage (optional for testing)
 */
// Uncomment to test retrieving all documents
/*
$documents = getAllDocuments();
echo "<h3>All Documents</h3><pre>";
print_r($documents);
echo "</pre>";
*/

// Uncomment to test retrieving a single document
/*

```

```

$doc = getDocumentById(2);
echo "<h3>Document with ID 2</h3><pre>";
print_r($doc);
echo "</pre>";
*/

```

As you can see, the AI assistant also provided guidance on retrieving all or just one document.

Let's go over some of the key features of the `getDocumentById` function:

- `$stmt = $pdo->prepare("...")` is a prepared statement with a named placeholder `:id`, which is used to query for a specific document securely.
- `$stmt->bindParam(':id', $id, PDO::PARAM_INT)` explicitly binds the `$id` parameter as an integer, enhancing security and type safety.
- `$document = $stmt->fetch(PDO::FETCH_ASSOC)` fetches a single row as an associative array. If no row is found, it returns `false`.

Having said this, let's start by uncommenting the section that retrieves all documents and see what we get.

Ensure you have the `crud.php` file in the `htdocs` folder, then open the browser and point it to `localhost/crud.php`.

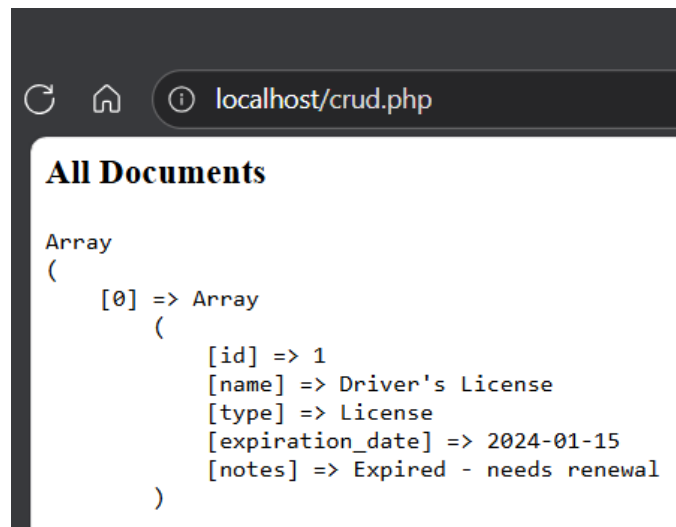


Figure 5-d: Retrieving All Documents (`crud.php`)

That worked! All the documents were retrieved (even though they are not shown in the previous figure).

Now, comment out the section that retrieves all documents and uncomment the section that retrieves only one document.

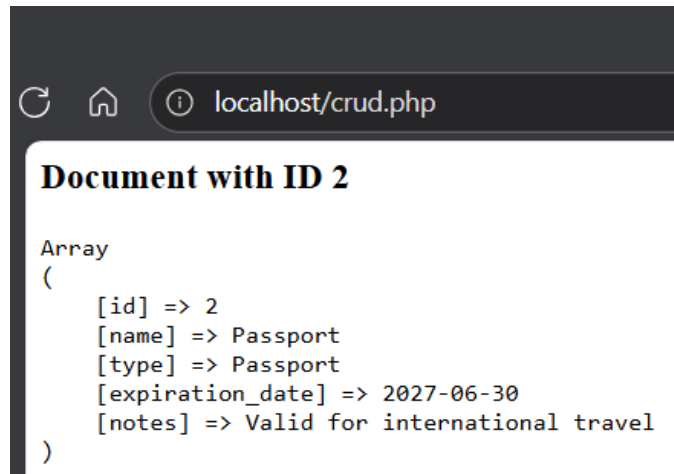


Figure 5-e: Retrieving One Document (crud.php)

Great—that also worked! Now, let’s expand crud.php for other operations.

Generating code for adding new documents (create)

Next, we need functionality to add new documents to our database. This will involve accepting data (likely from a web form) and securely inserting it into the documents table.

Code Listing 5-j: Second Prompt to Create a Consolidated CRUD File (crud.php)

Act as a PHP developer.

Expand crud.php and keep the existing and previous functionality of crud.php as is.

Write a PHP function named `addDocument(string $name, string $type, string $expirationDate, ?string $notes = null): bool` that inserts a new document record into the `documents` table.

The function should use prepared statements to prevent SQL injection and return `true` on successful insertion, `false` otherwise. Include proper error handling.

Add the ability to test this new functionality.

After executing this prompt, ChatGPT returns this result.

Code Listing 5-k: Second Prompt to Create a Consolidated CRUD File Result (crud.php)

Perfect – below is your expanded and production-ready crud.php file.

It keeps all existing functionality (getDbConnection(), getAllDocuments(), getDocumentById()) and adds a new function:

addDocument(string \$name, string \$type, string \$expirationDate, ?string \$notes = null): bool

- Inserts a new document securely using prepared statements.
- Returns true on success, false on failure.
- Includes robust error handling.
- Includes a test section at the bottom to verify insert functionality.

```
<?php
/**
 * crud.php
 * SQLite CRUD operations for the 'documents' table.
 */

ini_set('display_errors', 1);
error_reporting(E_ALL);

/**
 * Establishes a connection to the SQLite database.
 *
 * @return PDO
 * @throws Exception if the connection fails
 */
function getDbConnection(): PDO {
    try {
        $dbPath = __DIR__ . '/documents.sqlite';
        $pdo = new PDO("sqlite:" . $dbPath);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

/**
 * Retrieves all document records from the 'documents' table.
 *
 * @return array Returns an array of associative arrays (documents)
 * @throws Exception if the query fails
 */
function getAllDocuments(): array {
    try {
```

```

        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            ORDER BY expiration_date ASC
        ");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

/**
 * Retrieves a single document by its ID.
 *
 * @param int $id The document ID
 * @return array|null Returns the doc as an array, or null if not found
 */
function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            WHERE id = :id
            LIMIT 1
        ");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        $document = $stmt->fetch(PDO::FETCH_ASSOC);
        return $document !== false ? $document : null;
    } catch (PDOException $e) {
        error_log("Error fetching document by ID: " . $e->getMessage());
        return null;
    }
}

/**
 * Inserts a new document into the 'documents' table.
 *
 * @param string $name

```

```

* @param string $type
* @param string $expirationDate (in 'YYYY-MM-DD' format)
* @param string|null $notes
* @return bool True on success, False on failure
*/
function addDocument(string $name, string $type, string $expirationDate,
?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            INSERT INTO documents (name, type, expiration_date, notes)
            VALUES (:name, :type, :expiration_date, :notes)
        ");

        $stmt->bindParam(':name', $name, PDO::PARAM_STR);
        $stmt->bindParam(':type', $type, PDO::PARAM_STR);
        $stmt->bindParam(':expiration_date',
            $expirationDate, PDO::PARAM_STR);
        $stmt->bindParam(':notes', $notes, PDO::PARAM_STR);

        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error inserting document: " . $e->getMessage());
        return false;
    }
}

/**
 * Example usage for testing
 */

// Uncomment to test adding a new document
/*
$success = addDocument(
    "Business License",
    "License",
    "2026-09-10",
    "Issued by local government"
);
echo $success ? "Document added successfully!<br>" : "Failed to add
document.<br>";
*/

// Uncomment to test retrieving all documents

```

```

/*
$documents = getAllDocuments();
echo "<h3>All Documents</h3><pre>";
print_r($documents);
echo "</pre>";
*/

// Uncomment to test retrieving a single document by ID
/*
$doc = getDocumentById(2);
echo "<h3>Document with ID 2</h3><pre>";
print_r($doc);
echo "</pre>";
*/

```

As you can see, the response was very detailed, and all the previous functionality remains. The new functionality to add a document has been added and highlighted in yellow.

To test the new functionality, we can save this file as **crud.php** in the same directory as our `documents.sqlite` database file—within the `htdocs` folder. Then, uncomment the first test block at the bottom that invokes the `addDocument` function. Once done, open **localhost/crud.php** in the browser.

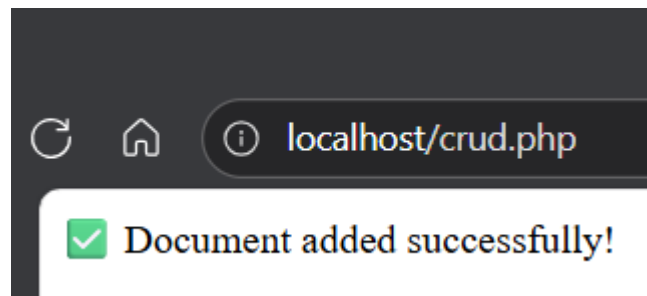
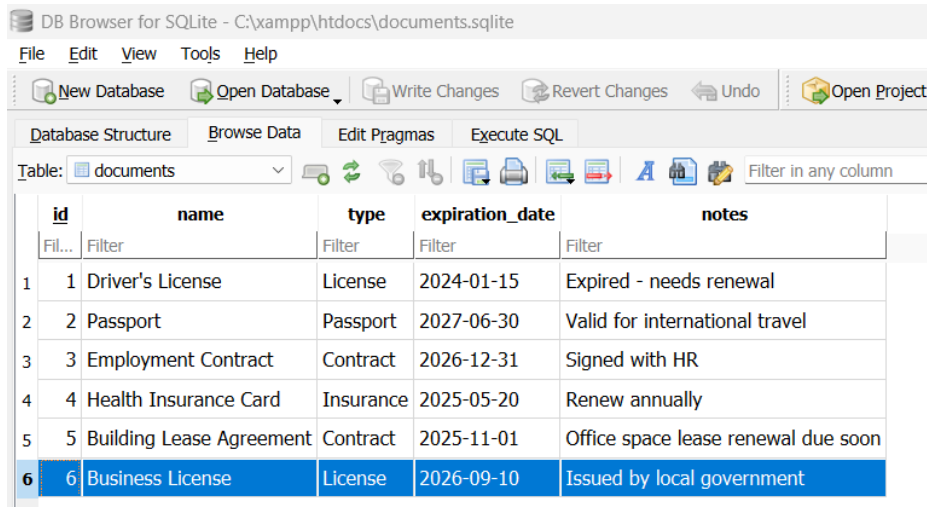


Figure 5-f: Document Added (crud.php)

If we now look in **DB Browser for SQLite**, we can see the new row added to the database table.



| | id | name | type | expiration_date | notes |
|---|----|--------------------------|-----------|-----------------|-------------------------------------|
| 1 | 1 | Driver's License | License | 2024-01-15 | Expired - needs renewal |
| 2 | 2 | Passport | Passport | 2027-06-30 | Valid for international travel |
| 3 | 3 | Employment Contract | Contract | 2026-12-31 | Signed with HR |
| 4 | 4 | Health Insurance Card | Insurance | 2025-05-20 | Renew annually |
| 5 | 5 | Building Lease Agreement | Contract | 2025-11-01 | Office space lease renewal due soon |
| 6 | 6 | Business License | License | 2026-09-10 | Issued by local government |

Figure 5-g: Document Added (DB Browser for SQLite)

Now, this worked out really well. You might notice that ChatGPT also sometimes offers the option to extend its functionality. In my case, it asked me the following.

Would you like me to extend this further to include `updateDocument()` and `deleteDocument()` next (so you have the full CRUD cycle)?

Figure 5-h: Follow-up Question from ChatGPT

This type of question is super handy because ChatGPT is well aware of the context and the style, and if I respond with a “yes,” I don’t have to re-explain the desired outcome. So, that’s exactly what I will do.

Generating code for updating and deleting

So, based on that positive response to ChatGPT’s last question, the following result has been returned.

Code Listing 5-l: Third Prompt to Create a Consolidated CRUD File (*crud.php*)

Perfect 🎉 – here’s the fully extended `crud.php` implementing all CRUD operations (Create, Read, Update, Delete) – while keeping everything from before intact.

It now includes:

-  `addDocument(...)` – Create a new document
-  `getAllDocuments()` – Get all documents
-  `getDocumentById($id)` – Get a single document
-  `updateDocument($id, ...)` – Update an existing document

-  deleteDocument(\$id) – Delete a document by ID

Each function:

- Uses prepared statements to prevent SQL injection.
- Includes error handling and logging.
- Returns appropriate values (true / false / array / null).

```
<?php
/**
 * crud.php
 * Complete SQLite CRUD operations for the 'documents' table.
 */

ini_set('display_errors', 1);
error_reporting(E_ALL);

/**
 * Establishes a connection to the SQLite database.
 *
 * @return PDO
 * @throws Exception if the connection fails
 */
function getDbConnection(): PDO {
    try {
        $dbPath = __DIR__ . '/documents.sqlite';
        $pdo = new PDO("sqlite:" . $dbPath);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        error_log("Database connection failed: " . $e->getMessage());
        throw new Exception("Unable to connect to the database.");
    }
}

/**
 * Retrieves all document records from the 'documents' table.
 *
 * @return array Returns an array of associative arrays (documents)
 */
function getAllDocuments(): array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
        ");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (Exception $e) {
        error_log("Error retrieving documents: " . $e->getMessage());
        return [];
    }
}
```

```

        ORDER BY expiration_date ASC
    ");
    $stmt->execute();
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    error_log("Error fetching documents: " . $e->getMessage());
    return [];
}
}

/**
 * Retrieves a single document by its ID.
 *
 * @param int $id The document ID
 * @return array|null Returns the document as an associative array, or
 * null if not found
 */
function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            SELECT id, name, type, expiration_date, notes
            FROM documents
            WHERE id = :id
            LIMIT 1
        ");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();
        $document = $stmt->fetch(PDO::FETCH_ASSOC);
        return $document !== false ? $document : null;
    } catch (PDOException $e) {
        error_log("Error fetching document by ID: " . $e->getMessage());
        return null;
    }
}

/**
 * Inserts a new document into the 'documents' table.
 *
 * @param string $name
 * @param string $type
 * @param string $expirationDate (YYYY-MM-DD format)
 * @param string|null $notes
 * @return bool True on success, False on failure

```

```

*/
function addDocument(string $name, string $type, string $expirationDate,
?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            INSERT INTO documents (name, type, expiration_date, notes)
            VALUES (:name, :type, :expiration_date, :notes)
        ");
        $stmt->bindParam(':name', $name);
        $stmt->bindParam(':type', $type);
        $stmt->bindParam(':expiration_date', $expirationDate);
        $stmt->bindParam(':notes', $notes);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error inserting document: " . $e->getMessage());
        return false;
    }
}

/**
 * Updates an existing document in the 'documents' table.
 *
 * @param int $id
 * @param string $name
 * @param string $type
 * @param string $expirationDate
 * @param string|null $notes
 * @return bool True on success, False on failure
 */
function updateDocument(int $id, string $name, string $type, string
$expirationDate, ?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("
            UPDATE documents
            SET name = :name, type = :type,
                expiration_date = :expiration_date, notes = :notes
            WHERE id = :id
        ");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->bindParam(':name', $name);
        $stmt->bindParam(':type', $type);
        $stmt->bindParam(':expiration_date', $expirationDate);
    }
}

```

```

        $stmt->bindParam(':notes', $notes);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error updating document: " . $e->getMessage());
        return false;
    }
}

/**
 * Deletes a document from the 'documents' table by ID.
 *
 * @param int $id
 * @return bool True on success, False on failure
 */
function deleteDocument(int $id): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("DELETE FROM documents WHERE id = :id");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error deleting document: " . $e->getMessage());
        return false;
    }
}

/**
 * -----
 * TEST SECTION (manual testing)
 * -----
 */

// Uncomment one block at a time to test specific functionality

// 1 Add a new document
/*
$added = addDocument("Software License", "License", "2027-03-15", "Annual
renewal required");
echo $added ? "☑ Document added successfully!\n" : "✗ Failed to add
document.\n";
*/

// 2 Get all documents
/*

```

```

$documents = getAllDocuments();
echo "<h3>All Documents</h3><pre>";
print_r($documents);
echo "</pre>";
*/

// 3 Get a single document by ID
/*
$doc = getDocumentById(2);
echo "<h3>Document with ID 2</h3><pre>";
print_r($doc);
echo "</pre>";
*/

// 4 Update a document
/*
$updated = updateDocument(2, "Updated Passport", "Passport", "2028-01-01", "Renewed recently");
echo $updated ? "✔ Document updated successfully!\n" : "✗ Failed to update document.\n";
*/

// 5 Delete a document
/*
$deleted = deleteDocument(5);
echo $deleted ? "✔ Document deleted successfully!\n" : "✗ Failed to delete document.\n";
*/

```

So, as you can see, it was a pretty comprehensive response, and all I had to do was answer with a “yes.”

To test this out, let’s first uncomment the bottom section, Update a document. Save the **crud.php** file to the **htdocs** folder and test it by navigating to **localhost/crud.php**.

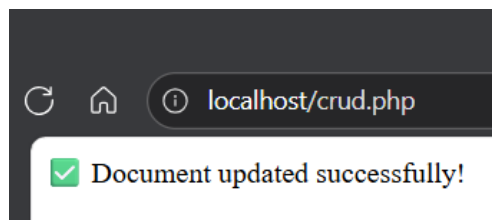


Figure 5-i: Document Updated (crud.php)

Excellent—so, the update-document functionality worked. Now, let’s comment out this section, uncomment the Delete a document section, and try it out.

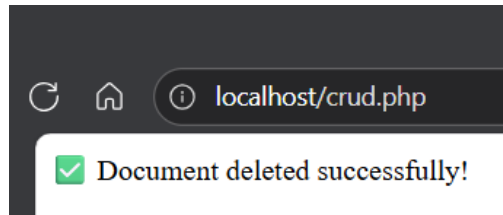


Figure 5-j: Document Deleted (crud.php)

As you can see, the delete-document functionality worked fine, too. Essentially, we have all the application's CRUD functionality ready.

Finished back-end code

Now that we know CRUD functionality works, let's ask the AI assistant to convert this code to fully functional back-end code that can be easily invoked from the application front end.

Let's use the following prompt to achieve that.

Code Listing 5-m: Turning the Consolidated CRUD File into a Back End

Make the crud.php file, with its existing functionality, behave like a proper back end, since I would like to add front-end functionality shortly.

This returns the following result.

Code Listing 5-n: Turning the Consolidated CRUD File into a Back-End Result (crud_api.php)

Excellent – let's turn your current crud.php into a RESTful back-end API for your document tracker, while preserving all existing CRUD functionality.

This version:

-  Handles GET, POST, PUT, and DELETE HTTP methods
-  Responds with JSON (perfect for front-end integration)
-  Uses the same SQLite DB and functions you already built
-  Includes error handling and proper HTTP status codes

Final crud_api.php (Back-End API Version)

Save this file as crud_api.php in the same directory as documents.sqlite.

```
<?php
/**
 * crud.php – REST API back end for the documents tracker
```

```

*/

ini_set('display_errors', 1);
error_reporting(E_ALL);

header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type, Authorization");

// Handle preflight request for CORS
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(200);
    exit;
}

/**
 * -----
 * DATABASE CONNECTION
 * -----
 */
function getDbConnection(): PDO {
    try {
        $dbPath = __DIR__ . '/documents.sqlite';
        $pdo = new PDO("sqlite:" . $dbPath);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        http_response_code(500);
        echo json_encode(["error" => "Database connection failed."]);
        exit;
    }
}

/**
 * -----
 * CRUD FUNCTIONS
 * -----
 */
function getAllDocuments(): array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("SELECT id, name, type, expiration_date,
            notes FROM documents ORDER BY expiration_date ASC");
    }
}

```

```

        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        error_log("Error fetching documents: " . $e->getMessage());
        return [];
    }
}

function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("SELECT id, name, type, expiration_date,
            notes FROM documents WHERE id = :id LIMIT 1");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();
        $document = $stmt->fetch(PDO::FETCH_ASSOC);
        return $document ?: null;
    } catch (PDOException $e) {
        error_log("Error fetching document by ID: " . $e->getMessage());
        return null;
    }
}

function addDocument(string $name, string $type, string $expirationDate,
    ?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("INSERT INTO documents (name, type,
            expiration_date, notes) VALUES (:name, :type, :expiration_date,
            :notes)");
        $stmt->bindParam(':name', $name);
        $stmt->bindParam(':type', $type);
        $stmt->bindParam(':expiration_date', $expirationDate);
        $stmt->bindParam(':notes', $notes);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error inserting document: " . $e->getMessage());
        return false;
    }
}

function updateDocument(int $id, string $name, string $type, string
    $expirationDate, ?string $notes = null): bool {
    try {

```

```

        $pdo = getDbConnection();
        $stmt = $pdo->prepare("UPDATE documents SET name = :name, type =
            :type, expiration_date = :expiration_date, notes = :notes
            WHERE id = :id");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->bindParam(':name', $name);
        $stmt->bindParam(':type', $type);
        $stmt->bindParam(':expiration_date', $expirationDate);
        $stmt->bindParam(':notes', $notes);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error updating document: " . $e->getMessage());
        return false;
    }
}

function deleteDocument(int $id): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("DELETE FROM documents WHERE id = :id");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        return $stmt->execute();
    } catch (PDOException $e) {
        error_log("Error deleting document: " . $e->getMessage());
        return false;
    }
}

/**
 * -----
 * API ROUTING
 * -----
 */
$method = $_SERVER['REQUEST_METHOD'];
$uri = explode('/', trim(parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH), '/'));
$resource = $uri[count($uri) - 1];
$id = isset($uri[count($uri) - 1]) && is_numeric($uri[count($uri) - 1]) ?
(int)$uri[count($uri) - 1] : null;

switch ($method) {
    case 'GET':
        if ($id) {
            $document = getDocumentById($id);

```

```

        if ($document) {
            echo json_encode($document);
        } else {
            http_response_code(404);
            echo json_encode(["error" => "Document not found."]);
        }
    } else {
        echo json_encode(getAllDocuments());
    }
    break;

case 'POST':
    $data = json_decode(file_get_contents("php://input"), true);
    if (!isset($data['name'], $data['type'],
        $data['expiration_date'])) {
        http_response_code(400);
        echo json_encode(["error" => "Missing required fields."]);
        exit;
    }
    $success = addDocument($data['name'], $data['type'],
        $data['expiration_date'], $data['notes'] ?? null);
    if ($success) {
        http_response_code(201);
        echo json_encode(["message" => "Document added
            successfully."]);
    } else {
        http_response_code(500);
        echo json_encode(["error" => "Failed to add document."]);
    }
    break;

case 'PUT':
    if (!$id) {
        http_response_code(400);
        echo json_encode(["error" => "Missing document ID."]);
        exit;
    }
    $data = json_decode(file_get_contents("php://input"), true);
    if (!isset($data['name'], $data['type'],
        $data['expiration_date'])) {
        http_response_code(400);
        echo json_encode(["error" => "Missing required fields."]);
        exit;
    }
}

```

```

    $success = updateDocument($id, $data['name'], $data['type'],
        $data['expiration_date'], $data['notes'] ?? null);
    if ($success) {
        echo json_encode(["message" => "Document updated
            successfully."]);
    } else {
        http_response_code(500);
        echo json_encode(["error" => "Failed to update document."]);
    }
    break;

case 'DELETE':
    if (!$id) {
        http_response_code(400);
        echo json_encode(["error" => "Missing document ID."]);
        exit;
    }
    $success = deleteDocument($id);
    if ($success) {
        echo json_encode(["message" => "Document deleted
            successfully."]);
    } else {
        http_response_code(500);
        echo json_encode(["error" => "Failed to delete document."]);
    }
    break;

default:
    http_response_code(405);
    echo json_encode(["error" => "Method not allowed."]);
    break;
}

```

So, how can we test this newly generated API code? We could do this by:

- Using the GET method to retrieve all the documents in /crud_api.php.
- Using the GET method to retrieve a specific document, e.g., /crud_api.php/2.
- Using the POST method to add a new document.
- Using the PUT method to update an existing document.
- Using the DELETE method to delete an existing document.

By refactoring this code, we now have a fully REST-compliant, CORS-enabled (cross-origin resource sharing) API that returns JSON responses and can be invoked from any front-end technology.

Once we generate the app's basic front end in the next chapter, we'll integrate the back end and test the API.

Chapter 6 App Front End

Overview

Alright, this is the part where you might be wondering whether to use one of those fancy code editors or vibe coding tools for the app's front end and UI—something like [Lovable](#), [v0](#), [Bolt](#), [Base44](#), or [Replit](#). There are many others.

Well, let me tell you something that most people don't know. Most of the vibe coding tools out there, at least the most well-known ones (previously listed), don't fully support PHP, and if they do, the results are somewhat limited—at least at the time of writing this ebook.

Maybe by the time you are reading this ebook, PHP support in some of these vibe coding platforms will be more prevalent.

Most of these tools support [React](#) and [Next.js](#) applications. Among the ones I tested, Replit has broader coverage and goes beyond React.

[PhpStorm](#) from JetBrains is a PHP-focused IDE that now includes AI-enhanced capabilities. I've not tried it myself, but I've heard from peers that it works quite well with PHP, and the AI features are pretty good. You might want to give this a try.

However, as I mentioned at the beginning of this ebook, the goal is to use something simple like ChatGPT to get results, so we'll sketch out the front end in the same way.

Where we left off

In the last prompt we used, ChatGPT returned the following.

Ready for Frontend Integration

This backend is now:

- Fully **REST-compliant**
- **CORS-enabled** (frontend JavaScript can call it)
- Returning **JSON responses**

Would you like me to now generate a **frontend dashboard** (e.g. simple HTML + JS or Vue/React) that connects to this backend and lists, adds, edits, and deletes documents?

Figure 6-a: The Last Part of the Previous ChatGPT Response

Notice how ChatGPT offers the option to generate the app's front end without requiring us to specify anything. I find this super cool and helpful.

So, let's go ahead and take advantage of ChatGPT's offer. Just a word of caution—don't expect the front-end UI to be something super fancy. It will probably be very bare bones. In any case, let's use the following prompt.

Code Listing 6-a: Initial Front-End Prompt

```
Yes, please generate a front-end dashboard with simple HTML and vanilla JavaScript (no Vue, no React, no Svelte) that connects to this back end, and lists, adds, updates, and deletes documents.

Essentially, add all the front-end CRUD functionality for the application using the existing and recently generated back-end API.

To make the UI look nice, please style using Bootstrap 5 classes. Looks are important, but functionality is more important.

Ensure that each CRUD front-end functionality works perfectly with its corresponding back-end functionality.
```

As you can see, the prompt is quite specific. Let's see what response we get back from our AI assistant.

Code Listing 6-b: Initial Front-End Prompt Result (index.html)

```
Perfect – below is a complete, ready-to-run front-end dashboard (index.html) that works seamlessly with your previously created PHP back end (crud.php).

☒ Highlights:



- Uses Bootstrap 5 for styling.
- Written in vanilla JavaScript (no frameworks).
- Supports full CRUD:
  - Read: Fetch and list all documents.
  - Create: Add a new document via a form.
  - Update: Inline edit for existing document fields.
  - Delete: Remove a document with confirmation.
- Automatically updates the table after every operation.



<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Document Dashboard</title>
  <link href="
https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
  rel="stylesheet">
```

```

</head>
<body class="bg-light">

  <div class="container py-4">
    <h1 class="text-center mb-4">📁 Document Management Dashboard</h1>

    <!-- Add Document Form -->
    <div class="card mb-4">
      <div class="card-header">Add New Document</div>
      <div class="card-body">
        <form id="addForm" class="row g-3">
          <div class="col-md-3">
            <input type="text" class="form-control" id="name"
              placeholder="Name" required>
          </div>
          <div class="col-md-3">
            <input type="text" class="form-control" id="type"
              placeholder="Type" required>
          </div>
          <div class="col-md-3">
            <input type="date" class="form-control" id="expiration_date"
              required>
          </div>
          <div class="col-md-3">
            <input type="text" class="form-control" id="notes"
              placeholder="Notes (optional)">
          </div>
          <div class="col-12 text-end">
            <button type="submit" class="btn btn-primary">Add
              Document</button>
          </div>
        </form>
      </div>
    </div>

    <!-- Documents Table -->
    <div class="card">
      <div class="card-header">Documents</div>
      <div class="card-body">
        <table class="table table-striped table-hover"
          id="documentsTable">
          <thead class="table-dark">
            <tr>
              <th>ID</th>

```

```

        <th>Name</th>
        <th>Type</th>
        <th>Expiration Date</th>
        <th>Notes</th>
        <th>Actions</th>
    </tr>
</thead>
<tbody></tbody>
</table>
</div>
</div>
</div>

<script>
    const API_URL = 'crud.php';
    const tableBody = document.querySelector('#documentsTable tbody');
    const form = document.getElementById('addForm');

    // Fetch and display all documents
    async function fetchDocuments() {
        const res = await fetch(API_URL);
        const data = await res.json();
        renderTable(data);
    }

    // Render table
    function renderTable(documents) {
        tableBody.innerHTML = '';
        documents.forEach(doc => {
            const tr = document.createElement('tr');
            tr.innerHTML = `
                <td>${doc.id}</td>
                <td contenteditable="true" data-field="name">${doc.name}</td>
                <td contenteditable="true" data-field="type">${doc.type}</td>
                <td contenteditable="true" data-
                    field="expiration_date">${doc.expiration_date}</td>
                <td contenteditable="true" data-field="notes">
                    ${doc.notes || ''}</td>
                <td>
                    <button class="btn btn-success btn-sm me-2"
                        onclick="updateDocument(${doc.id}, this)">Update</button>
                    <button class="btn btn-danger btn-sm"
                        onclick="deleteDocument(${doc.id})">Delete</button>
                </td>
            `;
        });
    }

```

```

        `;
        tableBody.appendChild(tr);
    });
}

// Add new document
form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const name = document.getElementById('name').value;
    const type = document.getElementById('type').value;
    const expiration_date =
        document.getElementById('expiration_date').value;
    const notes = document.getElementById('notes').value;

    const res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action: 'add', name, type,
            expiration_date, notes })
    });
    const result = await res.json();

    if (result.success) {
        form.reset();
        fetchDocuments();
    } else {
        alert('Error adding document: ' + result.error);
    }
});

// Update document
async function updateDocument(id, btn) {
    const row = btn.closest('tr');
    const updatedDoc = {};
    row.querySelectorAll('[data-field]').forEach(td => {
        updatedDoc[td.dataset.field] = td.innerText.trim();
    });
    updatedDoc.id = id;

    const res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action: 'update', ...updatedDoc })
    });
}

```

```

    const result = await res.json();

    if (result.success) {
        alert('Document updated successfully!');
        fetchDocuments();
    } else {
        alert('Error updating document: ' + result.error);
    }
}

// Delete document
async function deleteDocument(id) {
    if (
        !confirm('Are you sure you want to delete this document?')) return;

    const res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action: 'delete', id })
    });
    const result = await res.json();

    if (result.success) {
        fetchDocuments();
    } else {
        alert('Error deleting document: ' + result.error);
    }
}

// Initialize
fetchDocuments();
</script>
</body>
</html>

```

Expected Back End (crud.php)

Make sure your back end (crud.php) supports these actions:

- GET → Returns all documents as JSON.
- POST with JSON payload {action: 'add', ...} → Inserts a document.
- POST with JSON payload {action: 'update', ...} → Updates a document.

- POST with JSON payload `{action: 'delete', id: ...}` → Deletes a document.

If your current back end only supports `getAllDocuments()` and `getDocumentById()`, I can expand it to fully support add, update, and delete (REST-style) next.

Right off the bat, I noticed one error in the ChatGPT response. The back end is called `crud.php` instead of `crud_api.php`.

So, let's fix that by changing this `const API_URL = 'crud.php';` to `const API_URL = 'crud_api.php';`.

With that done, ensure the `index.html` file resides in the `htdocs` folder, then go to `localhost/index.html` to test it.

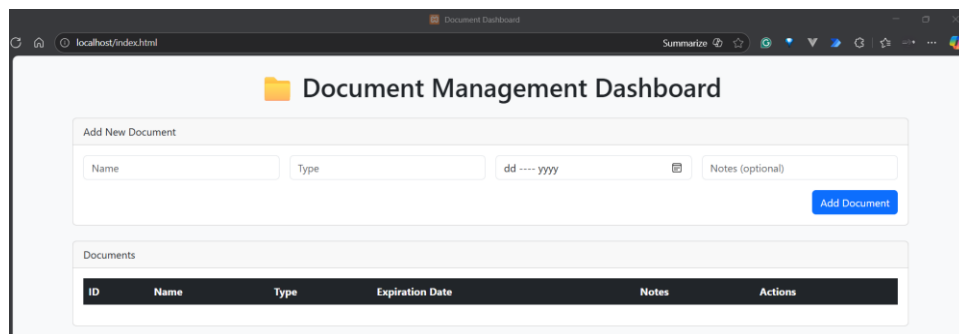


Figure 6-b: The Generated Front End

Look at that! Let's put this into context for a moment. It's not the prettiest site I've seen, but from a visual standpoint, it seems to be precisely what we need, at least for now.

The real test is whether this works with the back-end API. Because, from experience, I know this doesn't work right away. Therefore, I usually open the browser developer tools to catch anything that might happen.

I tried adding a new document and ran into the following issue. It's possible that you won't get this error at all, though.

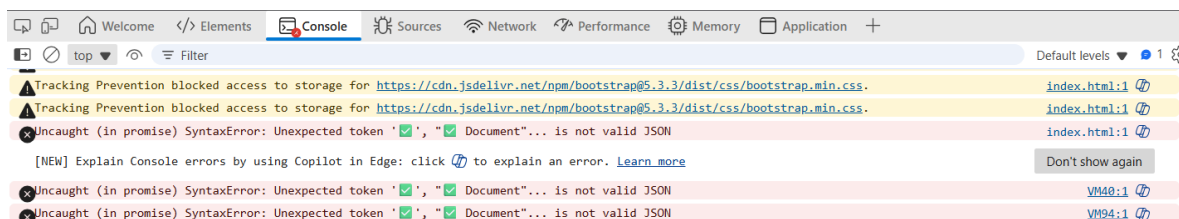


Figure 6-c: Errors (Browser Developer Tools)

To nail this down, I usually copy the error and paste it into the AI assistant chat window—in our case, ChatGPT. Let's do this and see what feedback we get.

Code Listing 6-c: ChatGPT Feedback

Perfect – that error tells us your PHP back end (crud.php) is printing human-readable messages like “☒ Document added successfully” instead of returning pure JSON, which breaks the front-end `fetch(...).json()` call.

Let’s fix that.

Below is a complete and correct crud.php back end that works seamlessly with the HTML dashboard I gave you earlier.

It:

- Uses PDO for SQLite (documents.sqlite).
- Returns proper JSON responses for all CRUD operations.
- Handles errors gracefully.
- Supports:
 - GET → Fetch all documents
 - GET?id=1 → Fetch single document
 - POST {action: "add"} → Add document
 - POST {action: "update"} → Update document
 - POST {action: "delete"} → Delete document

```
<?php
header('Content-Type: application/json');
error_reporting(E_ALL);
ini_set('display_errors', 1);

function getDbConnection(): PDO {
    try {
        $dbFile = __DIR__ . '/documents.sqlite';
        $pdo = new PDO('sqlite:' . $dbFile);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        echo json_encode(['success' => false, 'error' =>
            'Database connection failed: ' . $e->getMessage()]);
        exit;
    }
}

function getAllDocuments(): array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->query(
            "SELECT * FROM documents ORDER BY expiration_date ASC");
    } catch (PDOException $e) {
        echo json_encode(['success' => false, 'error' =>
            'Database connection failed: ' . $e->getMessage()]);
        exit;
    }
}
```

```

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        return [];
    }
}

function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("SELECT * FROM documents WHERE id = :id");
        $stmt->execute([':id' => $id]);
        $doc = $stmt->fetch(PDO::FETCH_ASSOC);
        return $doc ?: null;
    } catch (PDOException $e) {
        return null;
    }
}

function addDocument(string $name, string $type, string $expirationDate,
?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("INSERT INTO documents (name, type,
expiration_date, notes)
VALUES (:name, :type, :expiration_date, :notes)");
        return $stmt->execute([
            ':name' => $name,
            ':type' => $type,
            ':expiration_date' => $expirationDate,
            ':notes' => $notes
        ]);
    } catch (PDOException $e) {
        return false;
    }
}

function updateDocument(int $id, string $name, string $type, string
$expirationDate, ?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("UPDATE documents
SET name = :name, type = :type,
expiration_date = :expiration_date,
notes = :notes WHERE id = :id");
    }
}

```

```

        return $stmt->execute([
            ':id' => $id,
            ':name' => $name,
            ':type' => $type,
            ':expiration_date' => $expirationDate,
            ':notes' => $notes
        ]);
    } catch (PDOException $e) {
        return false;
    }
}

function deleteDocument(int $id): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("DELETE FROM documents WHERE id = :id");
        return $stmt->execute([':id' => $id]);
    } catch (PDOException $e) {
        return false;
    }
}

// -----
// REST-like request handling
// -----
$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
    if (isset($_GET['id'])) {
        $doc = getDocumentById((int)$_GET['id']);
        if ($doc) {
            echo json_encode($doc);
        } else {
            echo json_encode(['success' => false,
                'error' => 'Document not found']);
        }
    } else {
        echo json_encode(getAllDocuments());
    }
    exit;
}

if ($method === 'POST') {
    $input = json_decode(file_get_contents('php://input'), true);

```

```

if (!$input || !isset($input['action'])) {
    echo json_encode(['success' => false,
        'error' => 'Invalid JSON input']);
    exit;
}

switch ($input['action']) {
    case 'add':
        $ok = addDocument($input['name'],
            $input['type'], $input['expiration_date'],
            $input['notes'] ?? null);
        echo json_encode(['success' => $ok]);
        break;

    case 'update':
        if (!isset($input['id'])) {
            echo json_encode(['success' => false,
                'error' => 'Missing document ID']);
            break;
        }
        $ok = updateDocument((int)$input['id'], $input['name'],
            $input['type'], $input['expiration_date'],
            $input['notes'] ?? null);
        echo json_encode(['success' => $ok]);
        break;

    case 'delete':
        if (!isset($input['id'])) {
            echo json_encode(['success' => false,
                'error' => 'Missing document ID']);
            break;
        }
        $ok = deleteDocument((int)$input['id']);
        echo json_encode(['success' => $ok]);
        break;

    default:
        echo json_encode(['success' => false,
            'error' => 'Unknown action']);
    }
    exit;
}

// Default case

```

```
echo json_encode(['success' => false,
    'error' => 'Unsupported request method']);
```

Given this response, let's copy the generated code and overwrite the **crud_api.php** file, which should now look as follows.

Code Listing 6-d: crud_api.php (Updated)

```
<?php
header('Content-Type: application/json');
error_reporting(E_ALL);
ini_set('display_errors', 1);

function getDbConnection(): PDO {
    try {
        $dbFile = __DIR__ . '/documents.sqlite';
        $pdo = new PDO('sqlite:' . $dbFile);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $pdo;
    } catch (PDOException $e) {
        echo json_encode(['success' => false, 'error' =>
            'Database connection failed: ' . $e->getMessage()]);
        exit;
    }
}

function getAllDocuments(): array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->query(
            "SELECT * FROM documents ORDER BY expiration_date ASC");
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        return [];
    }
}

function getDocumentById(int $id): ?array {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("SELECT * FROM documents WHERE id = :id");
        $stmt->execute([':id' => $id]);
        $doc = $stmt->fetch(PDO::FETCH_ASSOC);
        return $doc ?: null;
    }
}
```

```

    } catch (PDOException $e) {
        return null;
    }
}

function addDocument(string $name, string $type, string $expirationDate,
?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("INSERT INTO documents (name, type,
expiration_date, notes)
VALUES (:name, :type, :expiration_date, :notes)");
        return $stmt->execute([
            ':name' => $name,
            ':type' => $type,
            ':expiration_date' => $expirationDate,
            ':notes' => $notes
        ]);
    } catch (PDOException $e) {
        return false;
    }
}

function updateDocument(int $id, string $name, string $type, string
$expirationDate, ?string $notes = null): bool {
    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("UPDATE documents
SET name = :name, type = :type,
expiration_date = :expiration_date,
notes = :notes WHERE id = :id");
        return $stmt->execute([
            ':id' => $id,
            ':name' => $name,
            ':type' => $type,
            ':expiration_date' => $expirationDate,
            ':notes' => $notes
        ]);
    } catch (PDOException $e) {
        return false;
    }
}

function deleteDocument(int $id): bool {

```

```

    try {
        $pdo = getDbConnection();
        $stmt = $pdo->prepare("DELETE FROM documents WHERE id = :id");
        return $stmt->execute([':id' => $id]);
    } catch (PDOException $e) {
        return false;
    }
}

// -----
// REST-like request handling
// -----
$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
    if (isset($_GET['id'])) {
        $doc = getDocumentById((int)$_GET['id']);
        if ($doc) {
            echo json_encode($doc);
        } else {
            echo json_encode(['success' => false,
                'error' => 'Document not found']);
        }
    } else {
        echo json_encode(getAllDocuments());
    }
    exit;
}

if ($method === 'POST') {
    $input = json_decode(file_get_contents('php://input'), true);
    if (!$input || !isset($input['action'])) {
        echo json_encode(['success' => false,
            'error' => 'Invalid JSON input']);
        exit;
    }

    switch ($input['action']) {
        case 'add':
            $ok = addDocument($input['name'],
                $input['type'], $input['expiration_date'],
                $input['notes'] ?? null);
            echo json_encode(['success' => $ok]);
            break;
    }
}

```

```

    case 'update':
        if (!isset($input['id'])) {
            echo json_encode(['success' => false,
                'error' => 'Missing document ID']);
            break;
        }
        $ok = updateDocument((int)$input['id'], $input['name'],
            $input['type'], $input['expiration_date'],
            $input['notes'] ?? null);
        echo json_encode(['success' => $ok]);
        break;

    case 'delete':
        if (!isset($input['id'])) {
            echo json_encode(['success' => false,
                'error' => 'Missing document ID']);
            break;
        }
        $ok = deleteDocument((int)$input['id']);
        echo json_encode(['success' => $ok]);
        break;

    default:
        echo json_encode(['success' => false,
            'error' => 'Unknown action']);
    }
    exit;
}

// Default case
echo json_encode(['success' => false,
    'error' => 'Unsupported request method']);

```

Your index.html file should look like the following.

Code Listing 6-e: index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Document Dashboard</title>

```

```

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/
dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container py-4">
  <h1 class="text-center mb-4">📁 Document Management Dashboard</h1>

  <!-- Add Document Form -->
  <div class="card mb-4">
    <div class="card-header">Add New Document</div>
    <div class="card-body">
      <form id="addForm" class="row g-3">
        <div class="col-md-3">
          <input type="text" class="form-control"
            id="name" placeholder="Name" required>
        </div>
        <div class="col-md-3">
          <input type="text" class="form-control"
            id="type" placeholder="Type" required>
        </div>
        <div class="col-md-3">
          <input type="date" class="form-control"
            id="expiration_date" required>
        </div>
        <div class="col-md-3">
          <input type="text" class="form-control"
            id="notes" placeholder="Notes (optional)">
        </div>
        <div class="col-12 text-end">
          <button type="submit"
            class="btn btn-primary">Add Document</button>
        </div>
      </form>
    </div>
  </div>

  <!-- Documents Table -->
  <div class="card">
    <div class="card-header">Documents</div>
    <div class="card-body">
      <table class="table table-striped table-hover"
        id="documentsTable">
        <thead class="table-dark">

```

```

        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Type</th>
            <th>Expiration Date</th>
            <th>Notes</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody></tbody>
</table>
</div>
</div>
</div>

<script>
    const API_URL = 'crud_api.php';
    const tableBody = document.querySelector('#documentsTable tbody');
    const form = document.getElementById('addForm');

    // Fetch and display all documents
    async function fetchDocuments() {
        const res = await fetch(API_URL);
        const data = await res.json();
        renderTable(data);
    }

    // Render table
    function renderTable(documents) {
        tableBody.innerHTML = '';
        documents.forEach(doc => {
            const tr = document.createElement('tr');
            tr.innerHTML = `
                <td>${doc.id}</td>
                <td contenteditable="true" data-field="name">${doc.name}</td>
                <td contenteditable="true" data-field="type">${doc.type}</td>
                <td contenteditable="true" data-field="expiration_date">
                    ${doc.expiration_date}</td>
                <td contenteditable="true" data-field="notes">
                    ${doc.notes || ''}</td>
                <td>
                    <button class="btn btn-success btn-sm me-2"
                        onclick="updateDocument(${doc.id}, this)">Update</button>
                    <button class="btn btn-danger btn-sm"

```

```

        onclick="deleteDocument(${doc.id})">Delete</button>
    </td>
    `;
    tableBody.appendChild(tr);
  });
}

// Add new document
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const name = document.getElementById('name').value;
  const type = document.getElementById('type').value;
  const expiration_date =
    document.getElementById('expiration_date').value;
  const notes = document.getElementById('notes').value;

  const res = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ action: 'add', name, type,
      expiration_date, notes })
  });
  const result = await res.json();

  if (result.success) {
    form.reset();
    fetchDocuments();
  } else {
    alert('Error adding document: ' + result.error);
  }
});

// Update document
async function updateDocument(id, btn) {
  const row = btn.closest('tr');
  const updatedDoc = {};
  row.querySelectorAll('[data-field]').forEach(td => {
    updatedDoc[td.dataset.field] = td.innerText.trim();
  });
  updatedDoc.id = id;

  const res = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },

```

```

        body: JSON.stringify({ action: 'update', ...updatedDoc })
    });
    const result = await res.json();

    if (result.success) {
        alert('Document updated successfully!');
        fetchDocuments();
    } else {
        alert('Error updating document: ' + result.error);
    }
}

// Delete document
async function deleteDocument(id) {
    if (!confirm(
        'Are you sure you want to delete this document?')) return;

    const res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action: 'delete', id })
    });
    const result = await res.json();

    if (result.success) {
        fetchDocuments();
    } else {
        alert('Error deleting document: ' + result.error);
    }
}

// Initialize
fetchDocuments();
</script>
</body>
</html>

```

Now, if you point the browser to **localhost/index.html**, you should see the list of existing documents contained in the SQLite database.

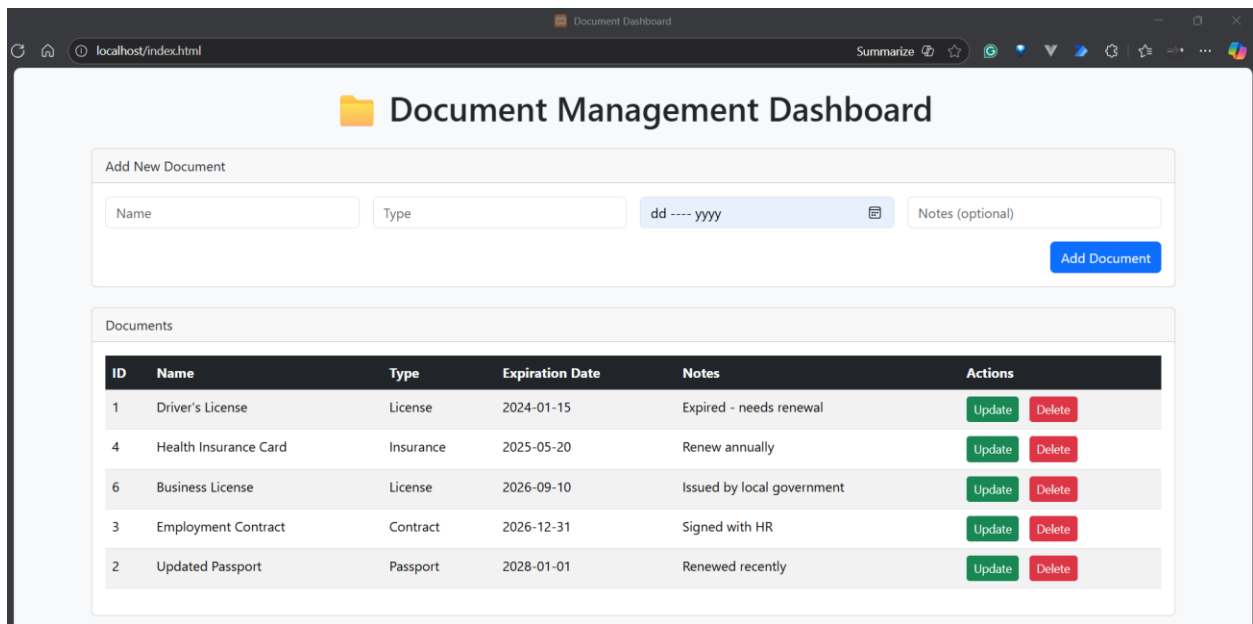


Figure 6-d: Document Management Dashboard with Various Documents

Great. Now, let's add a new document to test the back end.

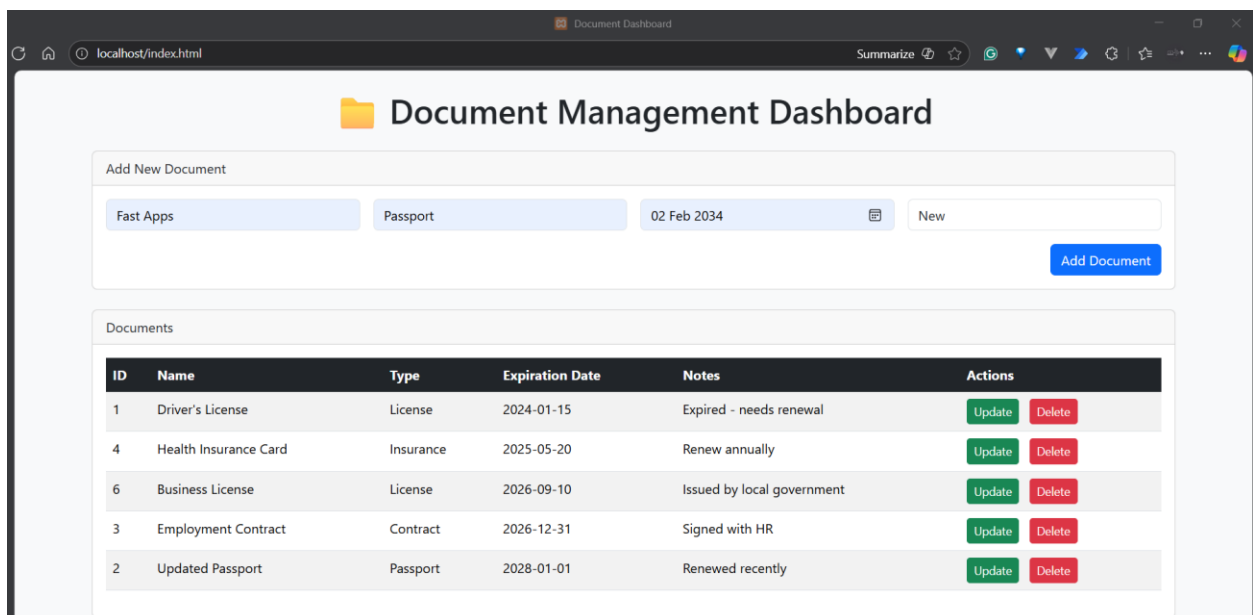


Figure 6-e: Adding a New Document

After clicking **Add Document**, we should see the new document added to the list.

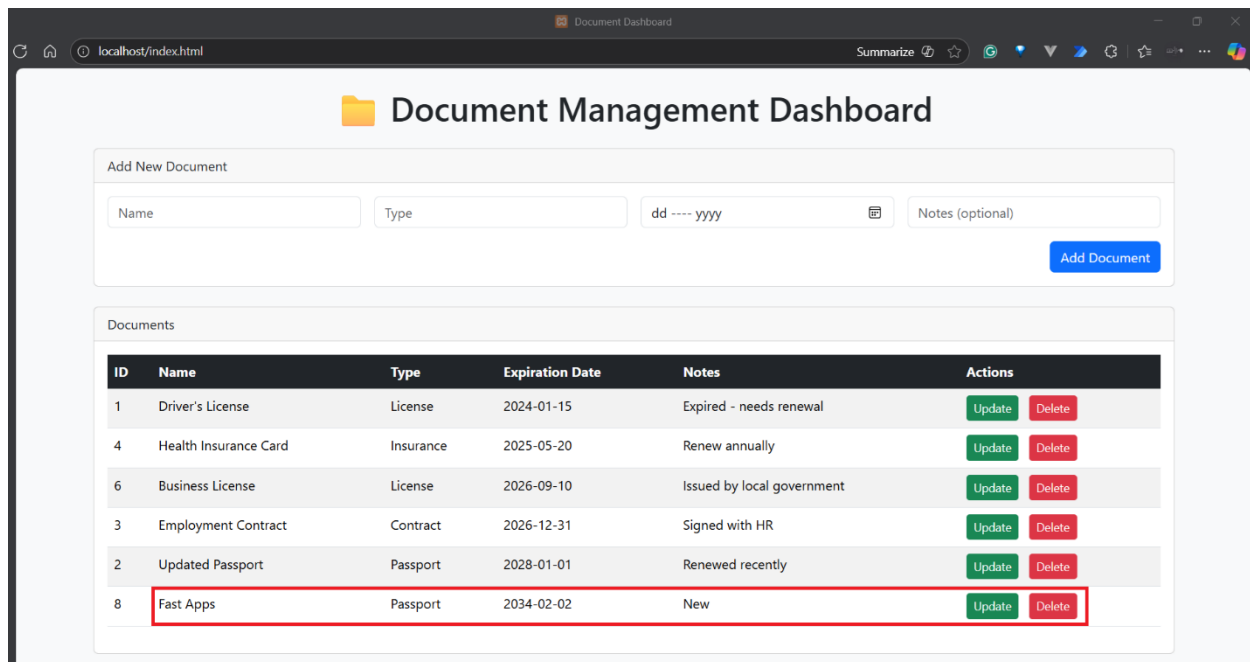


Figure 6-f: Document Added Successfully

Awesome. That also worked. Now, let's remove the document from the list by clicking **Delete** next to it.

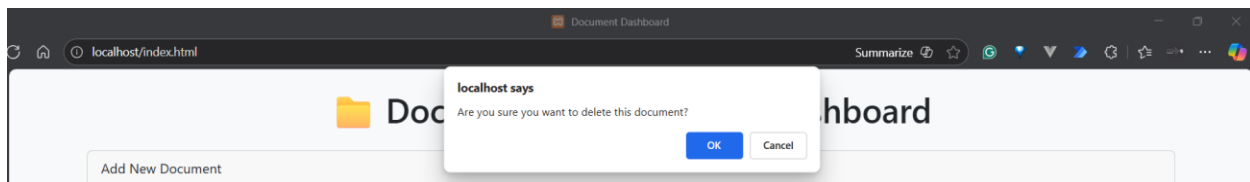


Figure 6-g: Requesting Confirmation to Delete

As you can see, the application prompts the user to confirm the deletion of the document. Let's click **OK**.

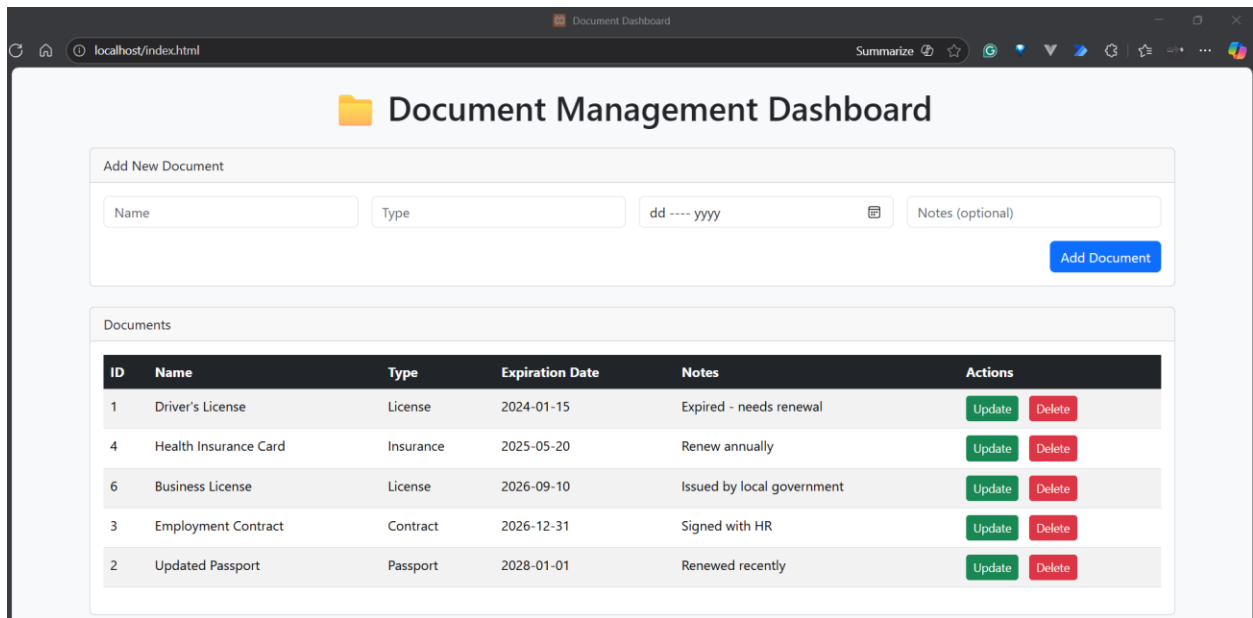


Figure 6-h: Document Deleted

After confirming, the document is successfully deleted. So, in essence, we have a working solution.

Closing Thoughts

Building this project was more than writing PHP and SQL; it was an exercise in refinement. Each prompt represented a step forward, a moment to test assumptions, correct direction, and evolve the code and design one iteration at a time.

We started with something simple: a function to fetch documents. Then we added structure, validation, error handling, and meaningful responses.

The back end took shape. The front end followed, lightweight, functional, and clean. Each piece became stronger because we didn't aim for perfection on the first try; we let clarity emerge through revision.

That's the real lesson here. Modern prompt engineering isn't a single act of creation; it's collaboration with your own reasoning, guided by feedback and iteration. Every improved prompt mirrored how we improve code: by tightening scope, naming things better, handling the edge cases, and seeing the system as a whole.

By the end, we achieved the complete solution—a working, secure, front-to-back web application. But more importantly, we had a record of how good software *evol/ves*. It's not magic. It's conversation, curiosity, and continuous refinement.

Keep prompting. Keep improving. Because in every iteration, the code—and you, the developer—get better. Until next time, take care, and keep learning.